Réf : 2014/ III/ Soutenu à la session de Juillet 2014

Université de la Manouba

ECOLE NATIONALE DES SCIENCES DE L'INFORMATIQUE



RAPPORT

de Mémoire de Fin d'Etudes

Présenté en vue de l'obtention du titre

d'INGENIEUR EN INFORMATIQUE

par

Aymen Kaabi

SUJET :

Réalisation d'un outil de calcul du temps de parcours d'une route avec la prise en compte d'un trafic prédictif

Organisme : CodersCloud

Nom du responsable : Monsieur Hichem Njeh

Nom du chef d'équipe : Monsieur Béchir Tourki

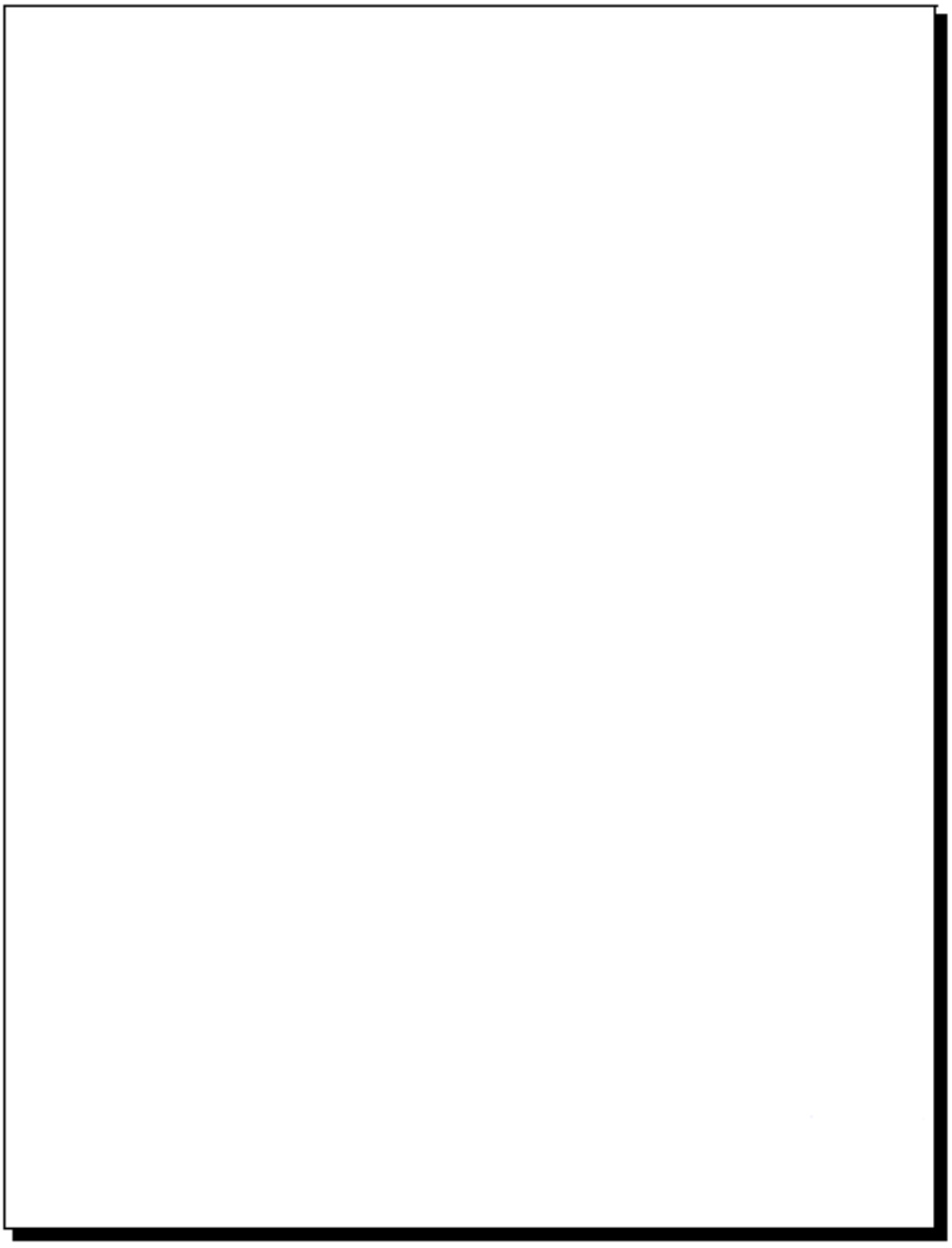
Encadré par : Monsieur Youssef Tourki

Supervisé par : Monsieur Mohamed Said Ouerghi

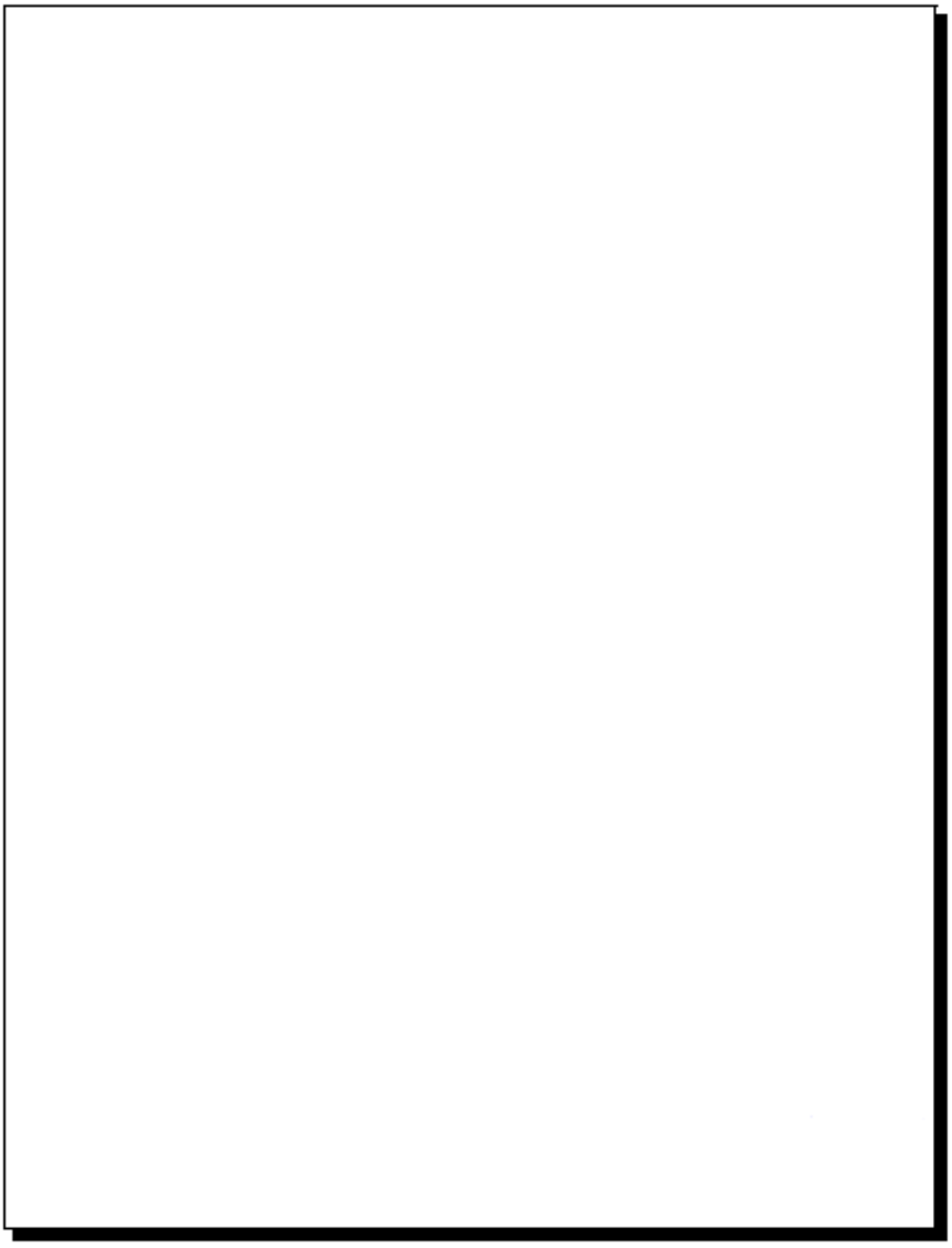
Adresse : Boulevard de la Terre, Tunis 1080

Téléphone : 52 297 691

Signature de l'Encadrant



Signature du Superviseur



Dédicaces

À mon père qui n'a jamais douté de moi, ni hésité à faire des sacrifices chaque fois qu'il était question de mon éducation et de mon avenir,

À ma mère pour les sacrifices qu'elle n'a cessé de fournir pour que je puisse être à la hauteur de ses ambitions,

À ma soeur qui m'a beaucoup aidé et soutenu tout au long de ce travail,

À mes deux frères qui n'ont cessé de m'encourager,

À tous mes amis et mes collègues,

Nous dédions ce mémoire.

Remerciments

Nous tenons à exprimer notre gratitude à nos encadrants Monsieur Béchir Tourki et Monsieur Youssef Tourki qui m'ont guidé d'aussi bonne grâce tout en prodiguant leurs précieux conseils le long de l'accomplissement de ce travail.

Je remercie, également, Monsieur Mohamed Said Ouerghi, enseignant à l'ENSI, pour m'avoir incités à mener à bien ce travail, pour son aide et ses précieux conseils.

Je remercie de même tout le personnel de CodersCloud et surtout le personnel du pôle Linkao pour leur générosité et leur soutien tout au long de la période de mon stage.

Enfin je remercions toute personne qui m'a, de loin ou de prés, accordé la faveur de son aide dans la réalisation de ce travail.

Résumé

Le présent mémoire a été rédigé dans le cadre du projet de fin d'études pour l'obtention du diplôme d'ingénieur en informatique. Ce projet a été effectué au sein de l'organisme CodersCloud. L'objectif de ce travail est détudier la faisabilité, de concevoir et de réaliser un service web qui permet davoir pour un trajet donné et une date future donnée une estimation du temps de parcours avec trafic.

Mots clés : VTC, Gretl, série temporelle, prédiction, cheminement, carte, graphe...

Abstract

The present report was written in the context of a final year project. This project is performed to obtain the computer science engineering diploma. The aim of this work is to study the feasibility, design and implement a web service that allows for a given path and a given future date to estimate travel time with traffic .

Key words : VTC, Gretl, time series, préediction, tracking, map, graph...

ملخص:

يندرج هذا العمل ضمن مشروع التخرج في المدرسة الوطنية لعلوم الإعلامية. ولقد تم إنجازه في شركة كودرز كلاود. ونحن نهدف من خلاله لتصميم منظومة لحجز السيارات الخاصة لمسافات قصيرة للتنقل مع الأخذ بعين الاعتبار الكثافة المرورية وإنجاز خارطة للمدينة لمعرفة أسرع الطرق. وبالتالي ان الهدف النهائي من هذا المشروع هو توفير خدمة بين أصحاب السيارات السياحية الخاصة وحرفاء سيارات الاجر توفر المعلومات اللازمة للحريف للوصول الى المكان المراد بأرخص ثمن وأسرع وقت وتقوي من مردودية سائقي السيارات السياحية.

Table des matières

Dédicaces **ii**

Remerciments **iii**

Table des figures **ix**

Liste des tableaux **xi**

Introduction **1**

Chapitre 1 Présentation Générale **3**

Introduction 3

1.1 Cadre Général 3

1.2 Organisme d'accueil 3

1.2.1 CodersCloud 3

1.3 Etude de l'existant 4

1.3.1 Service de voiture de tourisme avec chauffeur UBER 4

1.3.2 Service de voiture de tourisme avec chauffeur LeCab 6

1.3.3 Etude comparative 7

1.4 Objectif du projet 8

Conclusion 8

Chapitre 2 Etat De L'art **9**

Introduction 9

2.1 Cartographie en ligne (Web-Mapping) 9
2.1.1 Généralités 9
2.1.2 Fonctionnement du Web mapping 9
2.1.2.1 Le stockage 9
2.1.2.2 Le traitement 10
2.1.2.3 La diffusion 10
2.1.3 Le Datamining 11
2.1.4 Quelques APIs sig web en ligne 11
2.1.5 Google maps API 12
2.1.5.1 Google geocoding api 13
2.1.5.2 Google maps directions 13
2.1.6 Here maps API 13
2.2 Problèmes de cheminement dans un graphe 15
2.2.1 Définition d'un graphe 15
2.2.2 Algorithme de Dijkstra 16
2.2.3 Algorithme de Bellman-Ford 16
2.2.4 Algorithme A étoile 17
2.3 Introduction à la prédiction 17
2.3.1 La caractérisation des séries temporelles pour le choix de l'algorithme de prédiction 19
2.3.1.1 La stationnarité par époque 19
2.3.2 La méthode autorégressive 19
Conclusion 20

Chapitre 3 Analyse et spécification des besoins **21**

Introduction 21
3.1 Besoins fonctionnels 21
3.2 Besoins non fonctionnels 22
3.3 Système cible 22

3.4 Les acteurs 22
3.5 Spécification des besoins 22
3.5.1 Diagramme des cas d'utilisations 23
3.5.2 Génération d'un graphe automatique 23
3.5.3 Prédiction du trafic 25
3.5.4 Calcul du temps de parcours pour un chemin optimal 26
Conclusion 26

Chapitre 4 Conception **27**

Introduction 27
4.1 Conception globale 27
4.1.1 Architecture générale 27
4.1.2 Utilisation des services web Nokia Here et Google maps 28
4.1.3 L'architecture de l'environnement NodeJS 28
4.2 Conception détaillée 30
4.2.1 Etude de module de plus court temps de parcours d'un chemin . . . 30
4.2.1.1 Diagramme de classe 31
4.2.2 Etude de module de génération de graphe automatique 32
4.2.2.1 Diagramme de séquence 33
4.2.3 Etude de module de prédiction 34
4.2.3.1 Diagramme de séquence 35
Conclusion 36

Chapitre 5 Réalisation **37**

Introduction 37
5.1 Environnement de travail 37
5.1.1 Environnement matériel 37
5.1.2 Environnement Logiciel 38
5.1.3 Choix des langages 38
5.1.4 Choix technologiques 39

5.2 Implémentation 40
5.2.1 Génération d'un graphe automatique 40
5.2.2 Prédiction du trafic 44
5.2.3 Plus court temps de parcours 46
5.2.4 Gestion du projet 47

Conclusion et Perspectives **49**

Bibliographie **50**

Webographie **51**

Annexes **i**

Annexe A Problèmes de cheminement **i**

A.1 Algorithme de Dijkstra i
A.1.1 Principe de l'algorithme i
A.1.2 Description de l'algorithme i
A.2 Algorithme de Bellman-Ford ii
A.2.1 Principe de l'algorithme ii
A.2.2 Description de l'algorithme ii
A.3 algorithme A étoile iii
A.3.1 Les listes A étoile iii
A.3.2 Déroulement de l'algorithme iii

Table des figures

1.1 Réservation d'un taxi 5
1.2 Portail web du service LeCab 7

2.1 Architecture d'un serveur cartographique sur Internet 10
2.2 Affichage d'une route dans Google maps 14
2.3 Affichage d'une route dans Here maps 15
2.4 Graphe des villes de France 16

3.1 Diagramme des cas d'utilisations général 23
3.2 Diagramme des cas d'utilisations de génération d'un graphe automatique . 24
3.3 Diagramme des cas d'utilisations de prédiction du trafic routier 25
3.4 Diagramme des cas d'utilisations du calcul du temps de parcours 26

4.1 Architecture générale de l'application. 28
4.2 Architecture de l'environnement NodeJS 29
4.3 Architecture d'un serveur NodeJS 30
4.4 Architecture de module de plus court temps de parcours 31
4.5 Diagramme de classe de plus court temps de parcours 32
4.6 Architecture de module de génération de graphe automatique 33
4.7 Diagramme de séquence de génération d'un graphe 34
4.8 Architecture de module de prédiction 35
4.9 Diagramme de séquence de prédiction 35

5.1 Ville de Paris englobée dans un grand carré 40
5.2 Ville de Paris divisée en carré 41
5.3 Le fichier des arcs générés 41
5.4 La distribution des steps dans la carte 42
5.5 Nombre d'occurrences des steps 43

5.6 La distribution des steps importants dans la carte 43
5.7 Un sous graphe de la ville de Paris 44
5.8 Fichier d'une série temporelle 44
5.9 L'apprentissage des séries temporelles 45
5.10 La prédiction des valeurs futures 46
5.11 Page web de réservation d'un VTC 47
5.12 Détail de la résercation 47
5.13 Chronogramme des Tâches. 48

Liste des tableaux

2.1 Tableau comparatif des SIGs 12

5.1 Environnement matériel utilisé. 37

Glossaire

Ajax : Asynchrones JavaScript And Xml.

API : Application Programming Interface.

CSV : Comma Separated Values.

PHP : Pre-HyperTexte-Processor.

ASP : Active Server Pages.

GIF : Graphics Interchange Format.

GPS : Global Positioning System.

JSON : JavaScript Object Notation.

IGN : Institut Géographique National.

JPEG : Joint Photographic Experts Group.

JSP : JavaServer Pages.

NPM : Node Package Manager

PNG : Portable Network Graphics.

POI : point of interest.

SGBD : Système de Gestion de Bases de Données.

SIG : Système d'information géographique.

SVG : Scalable Vector Graphics.

SWF : ShockWave Flash.

VTC : Voiture de Tourisme avec Chauffeur.

UML : Unified Modeling Language.

XML : EXtensible Markup Language.

WGS84 : World Geodetic System 84.

Introduction

La mise en place d'un système de localisation mondial GPS a permis des progrès énormes dans le domaine militaire. L'amélioration de ce système coté précision et sa démocratisation au public, a créé des nouvelles opportunités pour les sociétés de services. Les clients mobiles sont les plus ciblés par ces services. Parmi eux nous citons les chauffeurs de taxi, les pilotes etc... .

Les clients et les chauffeurs de taxi ont beaucoup des problèmes concernant le temps de parcours et le trafic et aussi les réservations. Ces problèmes ont créé du dynamisme dans le marché européen de réseau de transport, avec plus de services et plus de performance. La société CLASS&CO SAS en partenariat avec la start-up CodersCloud vise à développer un service web qui remplit les critères exigées par les clients et les chauffeurs de taxi afin de mieux gérer le temps et permettre la réduction du coût.

Et comme solution à ces différents critères, la société CLASS&CO SAS a créé une branche pour étudier la faisabilité de concevoir et de réaliser un service web qui permet d'avoir pour un trajet donné et une date future donnée une estimation du temps de parcours avec trafic. Ce service dans un premier temps va être utilisé dans la ville de Paris (la grande couronne).

C'est dans ce cadre que s'inscrit notre projet qui consiste, en premier lieu, à concevoir et mettre en place une carte de la ville de Paris, en la transformant en un graphe des nœuds. En second lieu, il s'agit d'implémenter des algorithmes de graphe pour la détermination de plus court chemin en considérant le temps, le trajet .... En dernier lieu, il s'agit d'implémenter un module de prédiction pour l'estimation du temps de parcours.

Le présent rapport s'articule autour de cinq chapitres. Nous présenterons dans le premier chapitre le cadre du projet et l'organisme d'accueil, ainsi que le travail demandé. Dans le deuxième chapitre, nous présenterons une étude théorique approfondie sur la technologie web de web-mapping et les sciences mathématiques qui nous concerne, à savoir la prédiction et le problème de cheminement. Le troisième chapitre sera consacré à l'analyse et les spécifications des besoins.

Nous détaillerons les utilisations de l'application proposée à travers les diagrammes des cas d'utilisation. La conception fera l'objet du quatrième chapitre. Dans le dernier chapitre intitulé réalisation nous montrons l'environnement de travail matériel et logiciel ainsi que les technologies utilisés. Nous concluons notre rapport par un bilan du projet, ainsi que des perspectives.

Chapitre 1

Présentation Générale

Introduction

Dans ce chapitre, nous allons présenter brièvement notre projet. Tout d'abord nous définissons le cadre de sa réalisation, par la suite nous présentons l'organisme CodersCloud. Enfin nous introduisons le travail demandé.

1.1 Cadre Général

Notre travail s'inscrit dans le cadre du projet de fin d'étude ayant pour but l'enrichissement de notre formation et la mise en pratique des connaissances théoriques acquises. Ce projet, intitulé "Réalisation d'un outil de calcul du temps de parcours avec la prise en compte d'un trafic prédictif" est effectué au sein de la société CodersCloud, pour l'obtention du diplôme d'Ingénieur en Informatique.

1.2 Organisme d'accueil

1.2.1 CodersCloud

Depuis 2012, CodersCloud est basée entre Paris et Tunis. Elle a été créé par trois associés ayant fait les grandes écoles (Polytechnique, Mines ParisTech, Telecom ParisTech) et ayant travaillé pour des grands groupes français et internationaux (McKinsey, Glodman Sachs, Orange, Devoteam)....

CodersCloud est aussi une équipe jeune et dynamique d'une dizaine d'ingénieurs de haut niveau. Les équipes de développement sont basées à Tunis pour profiter du meilleur gisement de compétences francophones en terme de rapport qualité / prix. Les chefs de projets sont basés en région Parisienne pour une proximité client sans faille [N3].
Site Web : `http://www.coderscloud.com/`



1.3 Etude de l'existant

Il existe aujourd'hui et dans plusieurs villes du monde divers services de réservation de taxi offerts qui fonctionnent sur web et des smartphones comme par exemple Uber, LeCab, Taxiloc et Drive. Chaque service a ses caractéristiques qui répondent aux besoins des utilisateurs de taxi. En France, depuis un peu de temps, des entreprises spécialisées dans ce domaine commence à prendre place. Et comme tous les domaines, le début est difficile et plusieurs problèmes montent au surface. Ces problèmes créent une course entre les entreprises pour trouver des solutions et répondre aux attentes des clients. Le gagnant est celui qui prend le grand part du marché. Dans ce qui suit nous allons présenter deux services offerts sur Paris UBER et LeCab.

1.3.1 Service de voiture de tourisme avec chauffeur UBER

Uber est une start-up qui offre un service de voiture de tourisme avec chauffeur disponible sur smartphone mettant en relation les usagers du service et des conducteurs de voitures de luxe disponibles à la location. Le service permet de localiser, via le téléphone mobile, les véhicules les plus proches et de les réserver. Le service et l'application mobile associée sont développés par la société homonyme, Uber, basée dans la ville californienne de San Francisco. Uber est présent dans 22 pays et 62 villes [N4].

L'utilisateur réserve instantanément une voiture de tourisme avec chauffeur. Puis, il attend l'arrivée de la voiture réservé le plus proche du chauffeur qui accepte cette course. Le client peut aussi consulter le portail web `www.uber.com/cities/paris` pour savoir le mode de fonctionnement et la tarification.

La réservation se fait par une application Android ou Ios, il suffit d'ouvrir l'application et sélectionner l'option uberPOP. Une fois votre position confirmée, deux clics suffisent pour commander la voiture la plus proche de vous. Suivez son arrivée en temps réel directement dans votre application. Le tarif est calculé selon la durée et le trajet de course.

La figure 1.1 illustre la phase de réservation d'un taxi. Les voitures disponibles les plus proches sont affichées sur l'écran. Le client, après avoir commandé une voiture pour

une course, observe le déplacement de la voiture jusqu'à son arrivée.



FIGURE 1.1 – Réservation d'un taxi

Les problèmes du service Uber résident essentiellement dans :

- La facturation des courses : L'application uber calcule le tarif au fur et à mesure de la durée et du distance parcouru. A la fin du course un mail contenant la facture à payer. Le parcours d'une même course a plusieurs tarifs, ça dépend du chemin suivi et la vitesse de la voiture. Aussi, la marque de la voiture détermine la tarification à suivre.
- Réservation en avance : Le client ne peut que commander instantanément et il doit attendre un certain temps pour que le taxi arrive. La durée d'attente dépend

de la distance qui sépare le client et le taxi.

- le choix de chemin à parcouru :Le chauffeur est celui qui détermine le chemin à emprunter sans un outil qui l'aide à choisir le bon chemin à emprunter pour gagner du temps et du distance.

1.3.2 Service de voiture de tourisme avec chauffeur LeCab

LeCab est une start-up qui assure un service de voiture de tourisme avec chauffeur créée en décembre 2012 par Benjamin Cardoso. La commande peut être passée par aussi bien via leur site web `www.lecab.fr` que par téléphone ou via leur application (iPhone ou Android).

Une carte bancaire est exigée comme pour la plupart des VTC pour faire une réservation. La réservation se fait immédiatement ou à l'avance. Une fois la commande est passé, LeCab envoie alors un sms de confirmation avec le numéro de commande, le numéro de la plaque de votre chauffeur ainsi que le numéro de son portable en cas de problème. Ensuite, un deuxième sms de notification que le chauffeur est en route. Puis, un troisième sms de confirmation de l'arrivée de votre chauffeur [N5].

Le prix, fixé au moment de la commande pour éviter toute mauvaise surprise, comprend les frais d'approche, dix minutes d'attente et le trajet. Les tarifs sont très compétitifs, fonctionnent par zones, le prix aller sera le même que le prix du retour. Il est également possible de convenir d'un trajet avec une ou plusieurs étapes. Si le client souhaite effectuer une étape, non prévue initialement, le prix de la course sera alors recalculé et ajusté en fonction.

La figure 1.2 montre les différents services offerts sur le portail web de LeCab. Les prix, les commandes et les chauffeurs sont gérés par le site. Plusieurs informations sont disponibles aux usagers de ce service pour les bien renseigner. Des offres exceptionnelles sont mise à jour pour ses clients.

Les problèmes du service LeCab résident essentiellement dans :

- Le gain n'est pas garanti : Le service LeCab calcule les factures selon les distances parcourus. Ce qui fait que la facture ne prend pas en compte le temps de parcours que le chauffeur a dépensé pendant la course. Dans une heure ou le trafic est très dense le chauffeur sera le plus perdant. Aussi, si nous disons temps nous disons amortissement qui est n'est pas pris en compte dans cette formule.
- Prix fixe des courses usuels : Le client trouve que plusieurs prix des courses usuels (comme ceux à destination des aéroports) ne sont pas étudiés car ces prix dépassent les prix des taxis quand il n' y a pas de trafic dense. Donc, Le client



FIGURE 1.2 – Portail web du service LeCab

opte pour un taxi à compteur dans ses heures. Alors, les prix doivent prendre en compte le trafic pour qu'ils soient convenables dans toutes les heures.

- Réservation et gestion de flotte : Pour savoir gérer la flotte convenablement et respecter les dates de réservation il faut respecter le critère du temps. Ce critère n'est pas traité dans ce service . Pour bien répondre aux demandes des clients la société doit avoir une grande flotte. Donc, le service devient n'est pas pratique pour les chauffeurs à cause d'avoir moins de course.

1.3.3 Etude comparative

Après avoir étudié les deux services de VTC LeCab et Uber, nous passons à la réalisation d'une étude comparative de ces deux services. Nous commençons par le critère le plus attractif chez les clients de VTC , le prix. Dans les deux entreprises, le prix n'est pas étudié pour que le client et le chauffeur sortent gagnants tous les deux. LeCab fixe le prix avant le début de la course contrairement à Uber qui envoie la facture après la fin de course ce qui rend LeCAb plus avantageux pour les clients qui n'aiment pas les factures imprévues. Uber ne dispose pas de la réservation à l'avance ce qui rend toute la flotte disponible mieux que Lecab ce qui minimise la durée d'attente de voiture tandis que LeCab joue le plus sur la réservation à l'avance. LeCab offre des prix fixe pour certaines destinations et il fait des promotions selon les périodes. Concernant la prise en compte du

trafic, les deux entreprises ne tiennent pas compte jusqu'à maintenant de ce paramètre.

1.4 Objectif du projet

Pour remédier aux problèmes cités précédemment nous sommes invités à réaliser ce projet qui a pour objectif de réaliser un service web permet d'avoir pour un trajet donné et une date future donnée une estimation de le temps de parcours avec trafic.

Ce projet s'effectue en trois étapes :

1. **Création d'un graphe automatique pour la ville de Paris :**Cette étape sert à la préparation du terrain pour la création d'une structure adéquate qui permet d'appliquer les algorithmes du plus court chemin pour l'obtention d'une durée de parcours. Pour ce faire il faut tout d'abord assurer un bon choix des coordonnées pour la formation d'un graphe de la ville de Paris d'une manière automatique. Puis, les coordonnées doivent être affichées sur une carte pour la phase de validation.
2. **Déterminer le temps optimal de parcours en trafic entre deux coordonnées géographique dans la ville de Paris :** Cette étape consiste à concevoir et développer un outil qui prendre en paramètres les coordonnées deux points avec une date donnée qui se charge de créer un graphe avec les données qui correspondent à la date donnée puis nous faisons appel aux algorithmes de plus court chemin pour déterminer le temps de parcours optimal.
3. **Intégration d'un module de prédiction :** Cette étape consiste à créer un modèle statistique pour les séries temporelles des données du trafic Afin de permettre la prédiction du trafic pour le calcul du temps du parcours pour une date future.

Conclusion

Dans ce chapitre nous avons présenté l'organisme d'accueil et donné un aperçu sur le cadre du projet. Le chapitre suivant sera consacré à l'étude des différents concepts théoriques sur lesquels se base ce projet.

Chapitre 2

Etat De L'art

Introduction

Dans ce chapitre, nous allons explorer les axes de recherche qui ont été étudiés à travers ce projet. Notre politique de présentation vise l'assimilation du concept du point de vue théorique pour passer, en ayant des bases solides, à l'aspect pratique ou plutôt technique. Nous aborderons, dans un premier temps les services de cartographie en ligne. Puis nous présenterons le problème de cheminement dans un graphe et finalement les modèles statistiques .

2.1 Cartographie en ligne (Web-Mapping)

2.1.1 Généralités

Le Web mapping est représenté dans la publication des serveurs cartographiques (SIG) sur internet.
Un serveur cartographique SIG sur internet est un serveur Internet doté des fonctionnalités d'un véritable système d'information géographique. Un tel serveur se présente sous forme d'un logiciel installé sur une machine serveur web classique et permettant de répondre aux requêtes de type géographiques [N1].

2.1.2 Fonctionnement du Web mapping

Comme vous pouvez remarquer sur la figure 2.1, un serveur SIG possède les principales fonctions suivantes : le stockage, le traitement et la diffusion des cartes et informations géographiques via le réseau internet [B8].

2.1.2.1 Le stockage

Pour assurer sa fonction de stockage, le serveur SIG possède deux couches :

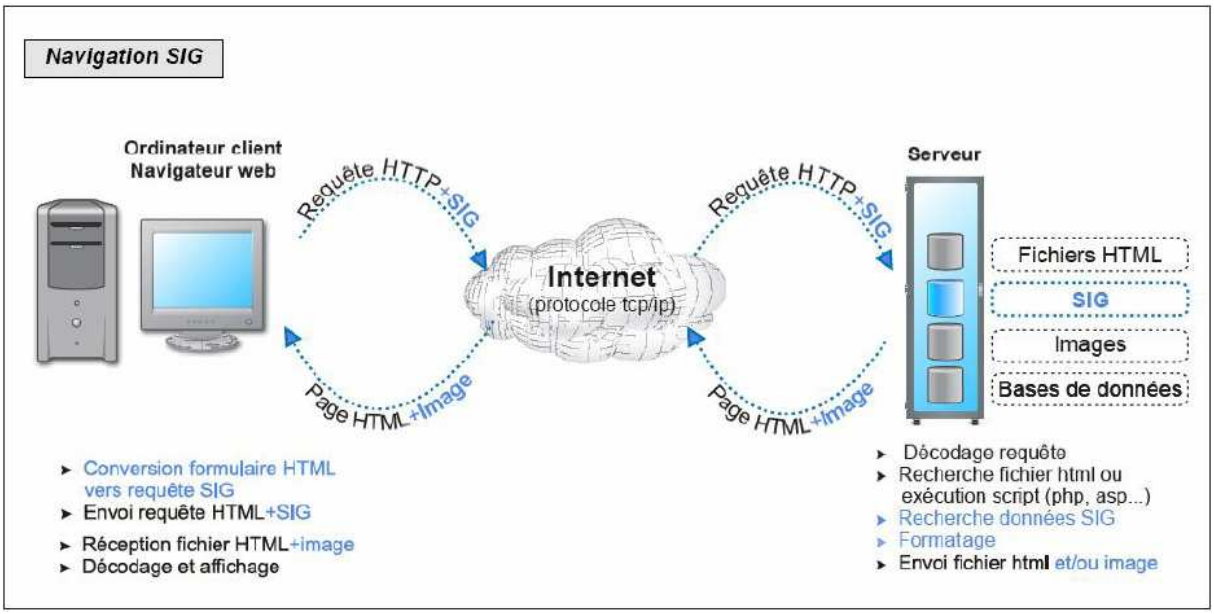


FIGURE 2.1 – Architecture d'un serveur cartographique sur Internet

Source : Serveur Cartographique et SIG interactifs en ligne M2 IASIG 2010

- Une couche dédiée aux données supportées par un SGBD géographique à l'instar de PostGresql avec son extension Post Gis, ou Oracle avec Oracle spatial etc...
- Des images géographiques référencées comme des cartes géographiques.

2.1.2.2 Le traitement

Pour le traitement, Un SIG Web est doté d'un serveur web, et d'un serveur de scripts (PHP, ASP, ou JSP) qui le rendent capable d'assurer les services de base d'un véritable moteur SIG, au-delà du stockage des données, c'est à dire la possibilité d'effectuer des requêtes à composante spatiale :

1. inclusion / juxtaposition / croisement spatial etc....
2. calculs de longueurs et superficies.
3. mesure de distances, zones tampons.
4. mise à jour des données graphiques et attributaires.
5. assemblage et habillage graphiques des couches d'information pour obtenir une carte.

2.1.2.3 La diffusion

Le serveur SIG va donc ajouter aux fonctions habituelles d'un serveur Internet des fonctions en relation avec la gestion et le traitement de données graphiques géo référencées. Cette fonction permettra aux utilisateurs indépendamment de leur situation géographique d'obtenir des informations géographiques sur un secteur de son choix. Enfin le serveur SIG

devra de ce fait être capable de retourner l'information sous une forme adaptée à l'interface Internet, c'est à dire au site affiché dans le navigateur de la machine cliente. Cela suppose en particulier la capacité de transformer les données graphiques et alphanumériques issues d'une requête dans le SGDB dans un format compatible avec les navigateurs : GIF, jpeg ou png pour les bitmaps et svg, swf (flash), pour les vecteurs (plugin nécessaire).

2.1.3 Le Datamining

Le data-mining est l'ensemble de techniques d'extraction d'informations d'ordre prévisionnel à partir de grandes bases de données. C'est un domaine connexe de l'analyse géo spatiale qui est encore assez peu passé dans l'usage courant des utilisateurs de SIG. Les outils de classification, de segmentation et de visualisation sont pourtant directement utiles pour un utilisateur des systèmes d'information géographique. Les SIG de par leur grande capacité à fédérer des masses importantes et diversifiées d'informations thématiques autour d'une carte géographique représentent une plateforme de développement du datamining.

2.1.4 Quelques APIs sig web en ligne

Depuis 2005, de nombreuses entreprises leader sur Internet comme Yahoo, Microsoft ,Nokia et Google se sont lancées dans le développement des services cartographiques en ligne accompagnés des API pouvant permettre aux développeurs d'intégrer dans leurs application SIG en ligne des fonds de carte provenant de leur source. Ces géants d'internet ont été joints dans leur initiative par certains organismes étatiques comme l'IGN en France et L'Ordonnance Survey au Royaumes Unis.
De plus en plus les services offerts par ces organismes permettent aux développeurs de représenter leurs propres données au-dessus des fonds de cartes servies et d'utiliser les nombreuses fonctions d'interactivités avec la carte mise à leur disposition. Ces fonctions couvrent la totalité des besoins de l'analyse spatiale (Zoom, réaction au click, calcul de distance, classification des phénomènes spatiaux etc. . .).

L'avantage de ces solutions, réside dans le fait qu'il n'y a plus nécessite d'installer un serveur de cartographie supplémentaire. C'est le navigateur qui exécute les fonctions d'affichage et d'interactivités cartographique grâce au JavaScript téléchargé depuis le serveur de ces organismes. Du coup le gain en temps d'affichage est remarquable par rapport aux serveurs de cartographie classiques.

Le tableau 2.1 comparatif montre les différentes fonctionnalités de ses quatre SIGs ce qui aide les utilisateurs à choisir le mieux. Dans notre cas Google API et Here API qui nous conviennent. Google API se démarque parmi tous ces géants des services cartographiques du fait de sa grande base de données spatiale réutilisable et de ses facilités

offertes aux développeurs et Here API est placé leader dans le domaine de données de trafic pour la création d'une base de données du trafic.

TABLE 2.1 – Tableau comparatif des SIGs

	Bing Maps	**Google Maps**	**Here**	**Yahoo Maps**
Société	Microsoft	Google	Nokia	Yahoo
En voiture	Oui	Oui	Oui	Oui
En transports en commun	Oui USA, UK	Oui USA	Oui	Non
En vélo	Non	Oui	Oui	Non
A pieds	Oui	Oui	Oui	Oui
Info-traffic	Oui	Oui	Non	Non
Orthophoto	Oui	Oui	Oui	Oui
Relief	Oui	Oui	Non	Oui
Routes	Oui	Oui	Oui	Oui
Cadastre	Non	Oui USA	Non	Non
Ordnance Survey Maps	Oui	Non	Non	Non
Collins Bartholomew Maps	Oui	Non	Non	Non
Vue à vol d'oiseau	Oui	Oui	Non	Non
Création de cartes	Oui	Oui	Non	
Ajout de POI	Oui	Oui	Oui	
Modification du fond de plan	Non	Oui	Non	
Plan de lieu	Oui	Oui	Non	Non
API	Oui	Oui	Oui	Non
Geocoding Web Service	Oui			
Directions Web Service	Oui		Oui	
Trafic Web Service		Oui	Oui	

2.1.5 Google maps API

Google maps est un serveur cartographique disponible sur le web à l'adresse `http://maps.google.fr`. Il couvre l'ensemble des données cartographiques mondiales. Google maps utilise comme système le WGS84 basé sur le GPS. Les utilisateurs du web peuvent interroger Google maps comme tout autre service de renseignement pour obtenir des informations géolocalisées sur une entreprise, un site touristique, une école etc..., Bref pour rechercher des lieux préalablement indexés sur la carte Google. L'efficacité de Google maps réside dans le fait que Google exploite sa technique de moteur de recherche classique avec Ajax pour rendre rapide et conviviaux l'affichage et les rafraîchissements des cartes sur le web. Google API comme nous l'avons vu au paragraphe précédent, offre une multitude de fonctions aux développeurs leurs permettant d'intégrer son fond de carte dans leur application et surtout d'interagir sur ces carte avec leur propres base de données attributaires. L'affichage d'une carte est déclenché par l'appel d'une fonction JavaScript portant les paramètres sur les dimensions du cadre d'affichage, le niveau de zoom, le point

central. Les données attributaires du client peuvent être également ajoutées à la carte sous forme de couche géographique par appel des fonctions JavaScript appropriés comme nous le verrons dans notre travail[N2].

2.1.5.1 Google geocoding api

L'API Geocoding de Google maps est très utile lorsque vous souhaitez récupérer des coordonnées GPS à partir d'une adresse (l'inverse étant également possible). Ce que facilite la tâche des développeurs pour cacher les détails aux utilisateurs qui connaissent pas préalablement qu'est-ce qu'une coordonnées géographique. L'appel de l'API Geocoding se fait côté client à l'aide d'une URL :

```
http://maps.googleapis.com/maps/api/geocode/json?address=\%s\&sensor=false
```

Par exemple, pour trouver les coordonnées GPS d'une adresse, l'URL sera :

```
http://maps.googleapis.com/maps/api/geocode/json?address=546\%20rue\%20Baruch\
%20de\%20Spinoza,\%20Avignon\&sensor=false
```

Cet appel vous renverra un résultat au format JSON qu'il sera possible d'exploiter par la suite.

2.1.5.2 Google maps directions

L'API Direction de Google maps permet essentiellement d'obtenir un chemin entre deux coordonnées GPS s'il existe. L'utilisation de l'api se fait soit par une requête au serveur directement via la barre de L'URL du navigateur soit par un code JavaScript implémenté dans une page web. La première méthode permet l'extraction des données relative au route entre un point de départ et un point d'arrivée, cette requête prend la forme ci-dessous et renverra un résultat au format JSON ou XML :

```
https://maps.googleapis.com/maps/api/directions/output?parameters
```

La deuxième méthode se fait par l'appel de l'api JavaScript de Google maps direction et en appelant les fonctions concernées on obtient une carte avec la route voulue dessinée. La figure 2.2 illustre le résultat d'exécution du code JavaScript coté client.

2.1.6 Here maps API

Here maps api est une API de service Web qui offre un acheminement facile et rapide pour plusieurs régions du monde. L'API offre les fonctionnalités suivantes :

- Calculer un itinéraire pour un ensemble de points d'intérêts.
- Mettre à jour un itinéraire précédemment calculé.
- Calculer un itinéraire isoline.

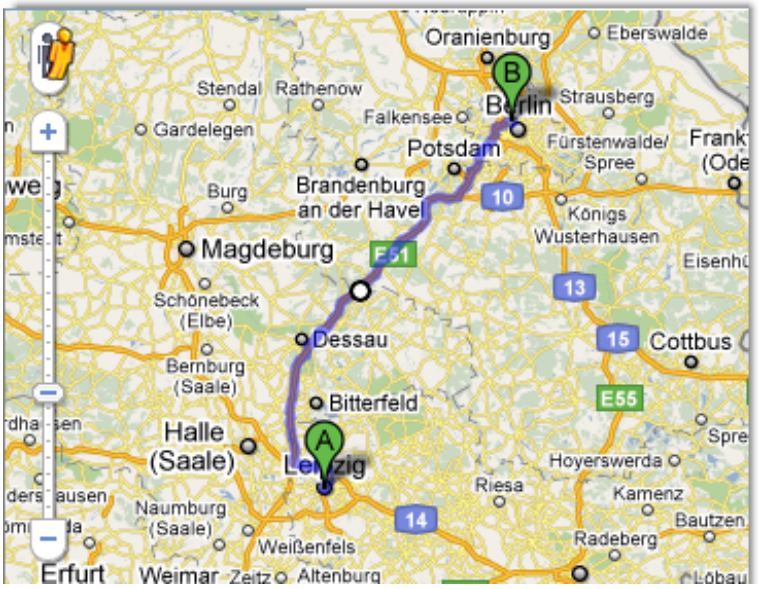


FIGURE 2.2 – Affichage d'une route dans Google maps

Here maps api est plus adapté au monde du transport. Il permet d'obtenir plusieurs informations nécessaires sur les routes, le trafic, les restrictions .... Parmi les informations, nous citons état de la route, classifications de réseau, passages bloqués, restrictions spéciales, manœuvres restreintes, routes à péage, catégories de vitesse [N6] .

Cet api est composé de quatre services principaux :
*le service calculate route : il permet de retourner un itinéraire entre deux points en se basant sur la formule du plus court chemin. L'appel de ce service se fait par une requête de cette forme `http://route.nlp.nokia.com/routing/6.2/calculateroute{format}?<parameter>=<value>` Les paramètres déterminent le mode utilisé (piéton, véhicule ...), le passage par les autoroutes, l'affichage des routes utilisés. Le format soit json soit xml.
*le service getroute : il permet de consulter un itinéraire déjà consulté avant par son id. La requête qui appelle ce service suit la forme suivante :
`http://route.nlp.nokia.com/routing/getroute.{format}?routeid=<ROUTEID>\&<parameter>=<value>`.
routeid représente l'id d'une route enregistré dans la base de here. Les paramètres sont les mêmes de ceux de service précédent.
*service getlinkinfo :il permet de retourner des informations détaillées sur une route. Nous pouvons savoir avec ce service les tronçons d'une route, dans quelle cité, dans quel département, sa longueur .... L'appel de ce service se fait par une requête de cette forme
`http://route.nlp.nokia.com/routing/6.2/getlinkinfo.{format}?<parameter>=<value>`...
*service Calculate Isoline :il permet de demander une poly ligne qui relie les points de toutes les routes sortant d'un centre défini avec soit une durée déterminée ou un temps de voyage spécifiée fin. L'appel de ce service se fait par une requête de cette forme
`http://route.nlp.nokia.com/routing/6.2/calculateisoline.{format}?<parameter>=<value>`...

La figure 2.3 montre une route entre le point de départ A et le point d'arrivée B en utilisant l'api here.

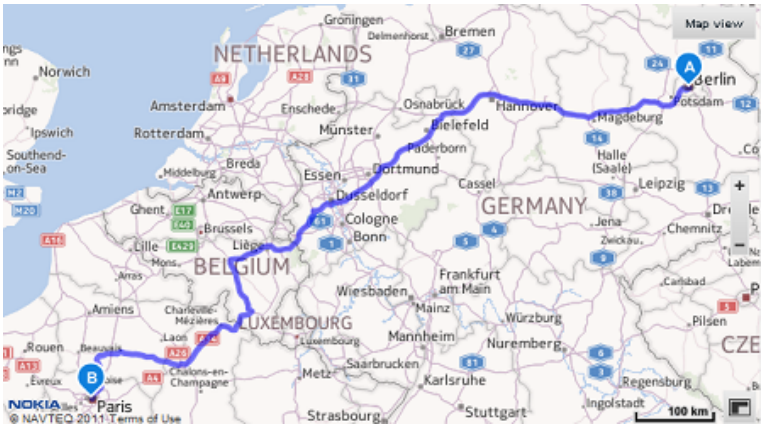


FIGURE 2.3 – Affichage d'une route dans Here maps

2.2 Problèmes de cheminement dans un graphe

Les problèmes de cheminement sont des problèmes classiques de la théorie des graphes. La résolution de ces problèmes permet de calculer un chemin entre des sommets d'un graphe qui minimise ou maximise une certaine fonction économique.

Le problème le plus classique consiste à chercher le chemin qui minimise la somme des valuations des arêtes traversées. Il existe des algorithmes polynomiaux pour résoudre ce problème, comme l'algorithme de Dijkstra et de Bellman-Ford.

2.2.1 Définition d'un graphe

Un graphe est un schéma constitué par un ensemble de points et par un ensemble de flèches reliant chacune deux de ceux-ci. Les pointsăsont appelés les sommets du graphe, et les flèches les arcs du graphe.

Un graphe G=(X,U) est le couple constitué :

1. par un ensemble X={ x_1 ,x_2....,x_n }
2. par une famille U={ u_1,u_2....,u_n } d'éléments du produit cartésien X x X={ (x,y)/x ∈ X,y ∈ Y } [N7]

La figure 2.4 illustre un graphe comportant villes de France comme des nœuds et les distances entre les villes comme des arcs.

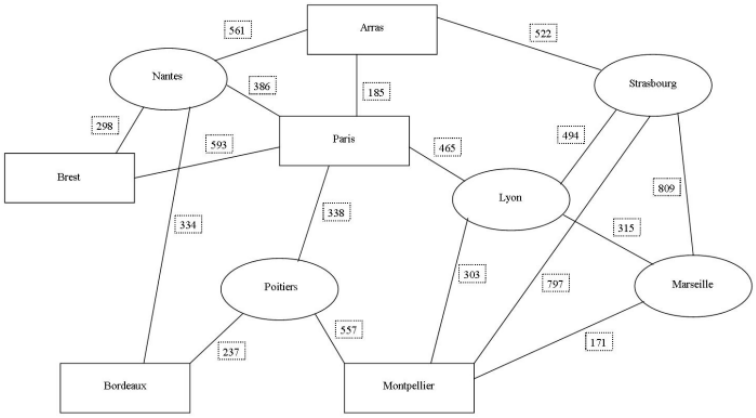


FIGURE 2.4 – Graphe des villes de France

2.2.2 Algorithme de Dijkstra

L'algorithme de Dijkstra sert à résoudre le problème du plus court chemin entre deux sommets d'un graphe connexe dont le poids lié aux arêtes est positif ou nul.
Pour illustrer l'intérêt de cet algorithme, on peut prendre pour exemple le réseau routier d'une région : Chaque sommet est une ville et chaque arc est une route dont le poids est le kilométrage. L'algorithme de Dijkstra permet alors de trouver le plus court chemin d'une ville à une autre.
L'algorithme porte le nom de son inventeur, l'informaticien néerlandais Edsger Dijkstra et a été publié en 1959. Cet algorithme est très largement utilisé, par exemple pour des applications de routage de données, de pathfinding dans des jeux vidéo et bien entendu de calcul d'itinéraires.
Plus en détail, cet algorithme retourne, à partir d'un sommet de départ, un tableau indiquant la distance jusqu'à chacun des autres sommets du graphe. Cependant, ce comportement n'est pas optimal dans notre application, et ce pour deux raisons : par exemple lorsqu'un parcours est trouvé jusqu'au sommet d'arrivée, Les parcours dont la distance est supérieure à cette distance sont inutiles. Il faudra donc modifier cet algorithme pour le rendre plus optimal [N8] (voir annexe A).

2.2.3 Algorithme de Bellman-Ford

L'algorithme de Bellmann-Ford est différent de l'algorithme de Dijkstra dans la mesure où il permet de traiter les graphes possédant des arêtes de poids négatif. De plus, il utilise les concepts de programmation dynamique et de programmation récursive pour optimiser les traitements.

Cet algorithme a été mis en place dès 1968 dans le ńRouting Information Protocolż (RIP) : protocole de routage IP qui a pour but d'optimiser l'échange de messages entre plusieurs routeurs sur le réseau Internet. En effet, il permet à chaque routeur de commu-

niquer aux autres routeurs la métrique (la distance qui sépare un routeur du réseau de destination), et ainsi de pouvoir gérer au mieux le circuit de transmission des messages [N9] (voir annexe A).

2.2.4 Algorithme A étoile

L'algorithme A étoile est une amélioration de l'algorithme de Dijkistra pour un chemin à source et destination connus. Il recherche de chemin dans un graphe entre un nœud initial et un nœud final. Il utilise une évaluation heuristique sur chaque nœud pour estimer le meilleur chemin y passant, et visite ensuite les nœuds par ordre de cette évaluation heuristique. C'est un algorithme simple, ne nécessitant pas de prétraitement, et ne consommant que peu de mémoire [N10].

A étoile commence à un nœud choisi. Il applique à ce nœud un coût (habituellement zéro pour le nœud initial). A étoile estime ensuite la distance qui sépare ce nœud du but à atteindre. La somme du coût et de l'évaluation représente le coût heuristique assigné au chemin menant à ce nœud. Le nœud est alors ajouté à une file d'attente prioritaire, couramment appelée open list(voir annexe A).

2.3 Introduction à la prédiction

Le principe fondamental de toute prédiction, c'est que l'avenir est déterminé par le passé. Si on veut prédire l'avenir, il faut observer le présent et connaître des lois qui déterminent l'avenir en fonction de son passé. La prédiction a montré son efficacité dans plusieurs domaines comme la météorologie, la finance et la biologie.

En vue d'optimiser les problèmes de prédiction, plusieurs travaux [B1, B2, B3, B4, B5] ont montré que la théorie d'apprentissage permet d'améliorer la précision des prédictions, construire de nouveaux algorithmes de prédiction, améliorer d'autres existants, etc.

Il existe généralement plusieurs types d'apprentissage qui se distinguent essentiellement par la nature de ce qui doit être appris. L'apprentissage supervisé permet de générer, à partir de l'ensemble d'apprentissage, un modèle f qui sert à prédire une variable cible sachant les valeurs des variables d'entrée. Cependant, l'apprentissage non supervisé permet de construire un modèle f* qui capture certains éléments de la véritable distribution ayant généré l'ensemble d'apprentissage. Ainsi, l'apprentissage non supervisé ne sert pas à générer des modèles de prédiction. Mais il peut être utilisé en amont de l'apprentissage supervisé en vue d'améliorer la performance du modèle à générer par ce dernier. D'une manière générale, il existe des méthodes de prédiction qui nécessitent une phase d'appren-

tissage et d'autres non.

Dans le cas d'un algorithme de prédiction nécessitant une phase d'apprentissage supervisé, par définition le processus général pour prédire l'évolution d'un système en cours du temps se déroule en trois principales étapes. La première étape consiste à analyser les séries temporelles du phénomène à prédire. La deuxième étape consiste à construire un modèle, selon une phase d'apprentissage supervisé, qui décrit la réalité de ce phénomène étant donné les analyses déduites de la première étape. Et la troisième étape consiste à faire des prédictions en utilisant ce modèle. L'objectif de l'apprentissage est d'automatiser ce processus de prédiction selon la nature de la méthode de prédiction adoptée. Cette définition est générale et peut être adaptée selon le problème de prédiction et le domaine d'application. Le système de prédiction résultant comporte une phase d'entraînement qui prend en entrée un ensemble de données appelé ensemble d'entraînement ou d'apprentissage et une phase de prédiction qui prend en entrée un ensemble des données de prédiction.

Dans le cas d'une méthode de prédiction qui ne nécessite pas d'apprentissage, l'algorithme de prédiction est appliqué directement [B6, B7] et le système de prédiction comporte seulement la phase de prédiction.

Les données d'entrée en amont du système de prédiction sont fournies sous forme de séries temporelles brutes exprimant les valeurs des variables d'entrée et la cible associée. Soit T la taille (le nombre de mesures) de ces séries temporelles. Les N premières valeur de ces séries temporelles forment l'ensemble des données d'entraînement. Les L (=T-N) valeurs restantes forment l'ensemble des données de prédiction. Les séries temporelles peuvent subir un prétraitement avant d'être utilisées par les algorithmes de prédiction et/ou d'apprentissage supervisé. Étant donné, les caractéristiques et la nature d'une série temporelle, certains prétraitements peuvent être nécessaires pour améliorer la précision des prédictions.

Le prétraitement de séries temporelles d'entrée consiste à les transformer pour répondre aux hypothèses de base de ces algorithmes tout en préservant les informations essentielles que contiennent-les données originales. Parmi les principales transformations qui peuvent constituer l'étape de prétraitement, nous pouvons citer le filtrage , la discrétisation et la normalisation .

Dans le contexte de la prédiction, la discrétisation ou encore la quantification peut être appliquée pour réduire le nombre des valeurs possibles (observées) et / ou le nombre d'échantillons dans les séries temporelles en vue d'améliorer la performance. Elle consiste à remplacer les séries temporelles à valeurs continues ou à valeurs dans un ensemble discret de grande taille par des valeurs d'un ensemble discret de taille assez plus petite.

Dans le cadre de la prédiction du trafic routier, la valeur de la variable cible est

estimée à partir d'une variable d'entrée représentée par le temps de parcours d'une route.

2.3.1 La caractérisation des séries temporelles pour le choix de l'algorithme de prédiction

Généralement, le choix d'une méthode de prédiction parmi plusieurs dépend principalement des caractéristiques statistiques des séries temporelles telles que la stationnarité , le chaos , l'auto similarité, l'auto-corrélation et le comportement par époque .

2.3.1.1 La stationnarité par époque

Par définition, une série temporelle $\{X_t\}_{t=1}^{T}$ (avec T le nombre des échantillons) est stationnaire au sens faible si les trois propriétés suivantes sont vérifiées :

1. L'espérance (notée E) est constante au cours du temps c'est-à-dire $E(\{X_i\}_{i=1}^{t}) = \mu < \infty$, pour tout t = 1, . . . , T,
2. La variance (notée Var) de chaque variable est constante et reste finie au cours du temps, c'est-à-dire $Var(\{X_i\}_{i=1}^{t}) = \sigma^2 < \infty$, pour tout t = 1, . . . , T,
3. iii. L'auto-covariance (noté Cov) et par conséquent l'autocorrélation entre la variable X_t et X_{t-k} dépend seulement de l'ampleur du décalage temporel k, c'est-à-dire $Cov(X_t, X_{t+k}) = Cov(X_{t-1}, X_{t-1+k}) = \ldots = Cov(X_0, X_k)$, pour tout t =1,. . . ,T, et k =1,. . . ,t.

Si au moins une des trois conditions précédentes est violée, les séries temporelles sont dites non stationnaires.
D'après la littérature, les modèles linéaires de prédiction (tels que ceux autorégressif et de moyenne mobile (voir section 2.3.2.) permettent des prédictions assez précises dans le cas de séries temporelles stationnaires et ne s'adaptent pas trop aux séries temporelles non stationnaires. Cependant les modèles non linéaires s'adaptent aussi bien aux séries temporelles stationnaires et non stationnaires mais sont plus complexes et nécessitent des temps de calcul plus long.

Par exemple, le trafic d'une route est toujours dense le matin, fréquemment léger pendant la nuit. Par conséquent, en appliquant les trois règles citées ci-dessus, nous trouvons que le trafic forme des séries temporelles stationnaires au sens faible.

2.3.2 La méthode autorégressive

En tenant compte seulement de l'autocorrélation pour prédire la valeur de la cible à l'instant de temps t en utilisant l'algorithme autorégressif AR(p, τ) uni-varié, $\hat{Y}_t$est

estimée à partir de p valeurs passées avec un décalage (pas) temporel de τ entre ces p valeurs selon le modèle de prédiction défini par l'expression suivante :

$\hat{Y_t}$=$\hat{X}_{t+1}$=$\sum_{j=1}^{p} a_j X_{t-\tau(j-1)} + erreur$.

Dans ce cas, la série temporelle est uni-variée autrement dit Y_t= X_{t+1}= $X_{1,t+1}$. Le plongement dans le passé est égal à l'ordre de prédiction p. Lors de la phase de l'apprentissage, il s'agit d'estimer les p coefficients a_j en vu de définir le modèle f. Ceci peut être fait en résolvant un système d'équations de Yule Walker. En tenant compte simultanément de l'autocorrélation et de la cross-corrélation en utilisant la méthode AR(p, τ) multi-variée, une estimation de la cible Yˆ_t est calculée selon le modèle de prédiction f défini par l'expression (2), à partir des p valeurs passées de chaque variable i = 1, . . . , M avec un décalage temporel τ :

$\hat{Y_t} = \hat{X}_{1,t+1} = \sum_{i=1}^{M}\sum_{j=1}^{p} a_{i,j} X_{i,t+1-\tau*(p-(j-1))} + erreur$ (2) Le plongement dans le passé de chaque variable i correspond à l'ordre de prédiction pi du modèle AR, pour tout i = 1, . . . , M. Par exemple, pour M égale à deux variables, la valeur de la cible est calculée des 2p valeurs les *plus* récentes [$X_{1,t+1-\tau}$, $X_{1,t+1-2\tau}$, $X_{1,t+1-3\tau}$, . . . , $X_{1,t+1-p\tau}$] et [$X_{2,t+1-\tau}$, $X_{2,t+1-2\tau}$, $X_{2,t+1-3\tau}$, . . . , $X_{2,t+1-p\tau}$] respectivement des variables {$X_{1,t}$ et $X_{2,t}$}. La phase d'apprentissage consiste à estimer les p coefficients $a_{i,j}$ de chaque ressource i = 1, . . . , M en vue de définir le modèle f. En appliquant l'espérance à l'expression (2) multipliée par $X_{i,t-k}$ pour tout k = 0, . . . , p, nous obtenons un sous-système de p+1 équations exprimé selon les fonctions d'autocorrélation de la ième variable et les fonctions de cross-corrélation entre cette ième variable et les autres variables. En faisant ce raisonnement pour chaque variable i = 0, . . . , M, nous obtenons un système de Yule-Walker comportant p.(M+1) équations linéaires. L'apprentissage supervisé consiste à : Estimer la moyenne de chaque variable i, estimer les pi fonctions d'autocorrélation de chaque variable i et les fonctions de cross-corrélation entre cette variable i et les autres variables. Résoudre le système des p.(M+1) équations. L'apprentissage supervisé de la méthode AR pure multi-variée, en se basant sur un ensemble d'entrainement comportant N données, nécessite un temps de calcul de l'ordre de sup[O(N.p.(M+1)) , O[p(M+1)].

Conclusion

Tout au long de ce chapitre, nous avons présenté les grands traits de notre projet. Nous avons ainsi mis en relief les points critiques à soigner et dégagé les parties à aborder lors des étapes suivantes de l'implémentation de notre solution.

Dans le chapitre suivant, nous analyserons notre logiciel pour spécifier les principales fonctionnalités de l'application à réaliser.

Chapitre 3

Analyse et spécification des besoins

Introduction

Le but de ce chapitre est de définir et d'analyser les différents besoins et caractéristiques de la future application. Nous allons nous concentrer sur les fonctionnalités requises par les utilisateurs. Elles seront présentées dans un premier temps d'une manière textuelle et dans un second temps elles seront détaillées à l'aide du langage de modélisation UML avec des diagrammes des cas d'utilisation.

3.1 Besoins fonctionnels

L'analyse fonctionnelle est une démarche qui consiste à rechercher et à caractériser les fonctions offertes par un produit pour satisfaire les besoins de son utilisateur. En effet, ils expriment les attentes des différents utilisateurs envers le système.

Nous désignons par les besoins fonctionnels les différentes fonctionnalités qui seront assurées par le système et qui seront offertes aux différents acteurs. Dans ce qui suit, nous nous intéressons qu'aux fonctionnalités qui touchent le cœur du système :

- Créer un graphe automatique de la ville de Paris (grande couronne) qui prend en considération tous les points géographiques importants que les chauffeurs de taxi les utilisent en permanence.
- Extraire des données relatives aux arcs du graphe en créant des séries temporelles de trafic à des intervalles égaux.
- Mettre en palce un service web qui calcule pour deux points géographiques dans Paris le chemin optimal en prenant en compte le temps de parcours et la longueur de chemin.
- Afficher les nœuds parcourus.
- Générer un modèle de prédiction de trafic.
- Prédire le trafic.

3.2 Besoins non fonctionnels

En plus des besoins fonctionnels, notre système doit respecter certaines contraintes nécessaires pour le bon fonctionnement de l'application, à savoir :

1. Le temps de réponse et la rapidité d'accès aux différentes données doivent être pris en considération.
2. Le service web doit supporter la surcharge des requêtes en réseau et répondre dans un délai ne dépasse pas trois secondes.
3. Le développement du serveur sera sous le système linux. Il doit assurer le même fonctionnement sur Windows.

3.3 Système cible

Notre projet cible les services de Voiture de tourisme avec chauffeur afin de trouver des solutions pour les problèmes de transport. Notre projet vise à fournir les outils nécessaires pour créer un graphe pour n'importe quelle ville dans le monde. Ces outils facilitent la tâche des développeurs pour ne pas commencer à zéro pour chaque projet dans une ville. La deuxième partie de notre projet cible les chauffeurs des VTC et les taximans et les clients de taxi principalement. Elle offre un service qui cherche un plus court chemin en prenant compte du trafic pour un trajet d'une course. La troisième partie concerne les réservations futures des clients. Cette phase nécessite un service de prédiction qui est assuré par notre projet.

3.4 Les acteurs

après avoir présenté les services réalisés par notre projet, nous passons à définir les acteurs de notre système :

- Développeur : utiliser le module de génération de graphe automatique d'une ville.
- Administrateur : utiliser le module de prédiction pour préparer les données futures à consulter par les utilisateurs.
- Internaute : entrer l'origine et la destination de son chemin avec la date et l'heure pour consulter le temps de parcours et le chemin à emprunter.

3.5 Spécification des besoins

Après l'analyse et le recensement des besoins fonctionnels et non fonctionnels, ces données sont regroupées afin de décrire clairement ces besoins par un diagramme des cas d'utilisation.

3.5.1 Diagramme des cas d'utilisations

Afin de décrire les exigences fonctionnelles du système, nous faisons appel au diagramme des cas d'utilisation. Ce dernier modélise les services rendus par le système et exprime les interactions entre les acteurs et le système.

Notre application se divise en trois modules complémentaires. Le premier est celui de génération du graphe automatique, le deuxième est celui de prédiction du trafic et le troisième celui du calcul du temps de parcours de chemin optimal. Nous pouvons proposer le diagramme des cas d'utilisations présenté par la figure 3.1.

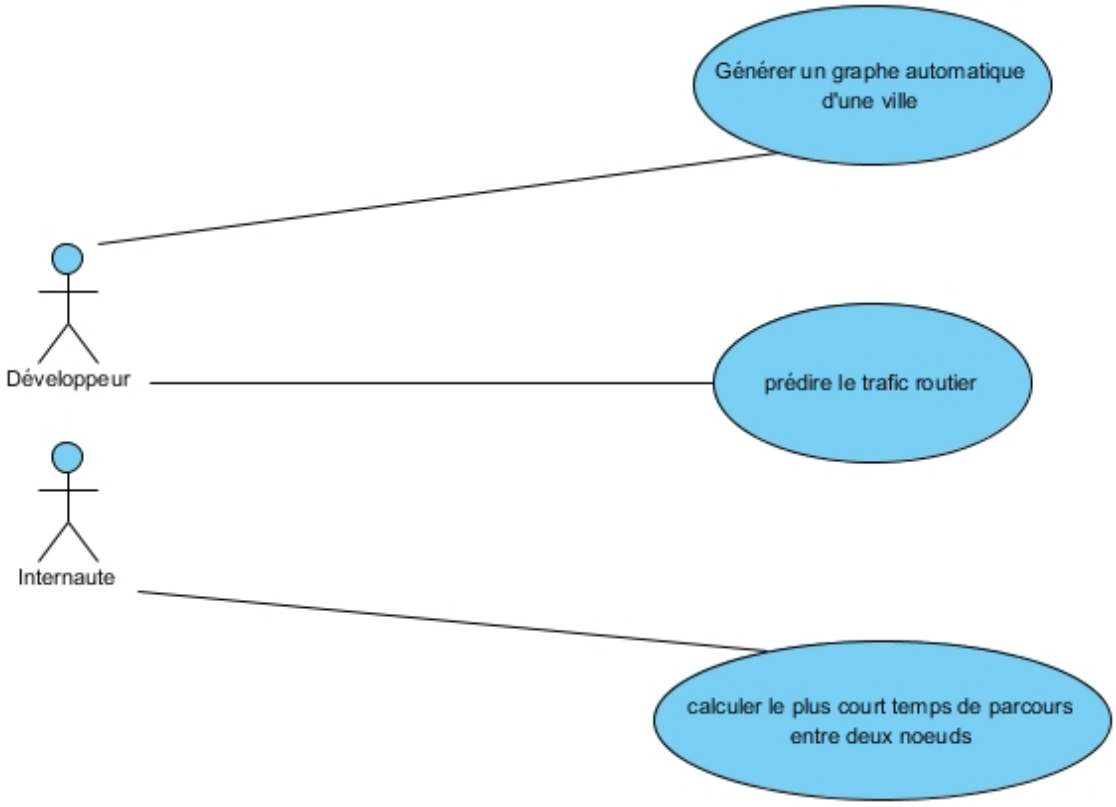


FIGURE 3.1 – Diagramme des cas d'utilisations général

3.5.2 Génération d'un graphe automatique

Dans un premier temps, le développeur a besoin des outils nécessaires pour la génération d'un graphe automatique pour l'afficher par la suite sur une carte. Donc, l'acteur principal est le développeur. Le diagramme des cas d'utiliation suivant représenté par la figure 3.2 met l'accent sur les différentes fonctionnalités qui réalisent ce module.

La génération d'un graphe automatique peut se résumer aux tâches suivantes :

- Englober la ville dans un carré composé de petits carrés : la ville est divisé en des petits carrés (diviser pour régner) et enregistrer leurs sommets.
- Supprimer les sommets qui n'appartiennent pas à la ville : nous intéressons qu'aux points appartenant à la ville.
- Lier les sommets proches : former des arcs entre les sommets proche .
- Déterminer les steps entre les sommets proches : nous cherchons les routes des arcs puis nous extrayons les steps(les points qui définissent une route).
- Trier les steps par leur nombre d'occurrence : les steps sont liés à plusieurs routes donc nous allons avoir beaucoup des occurrences.
- Déterminer les nœuds du graphe : les steps qui ont un nombre d'occurrence important sont choisis comme des nœuds car le nombre d'occurrence montre que cet step est un point clé pour la route et représente un point d'utilisation importants par les chauffeurs.
- Former le graphe : après avoir déterminé les nœuds du graphe, nous créons des arcs entre les nœuds proches en suivant les routes (pas les distances).

La figure 3.2 décrit le comportement fonctionnel de génération d'un graphe automatique.

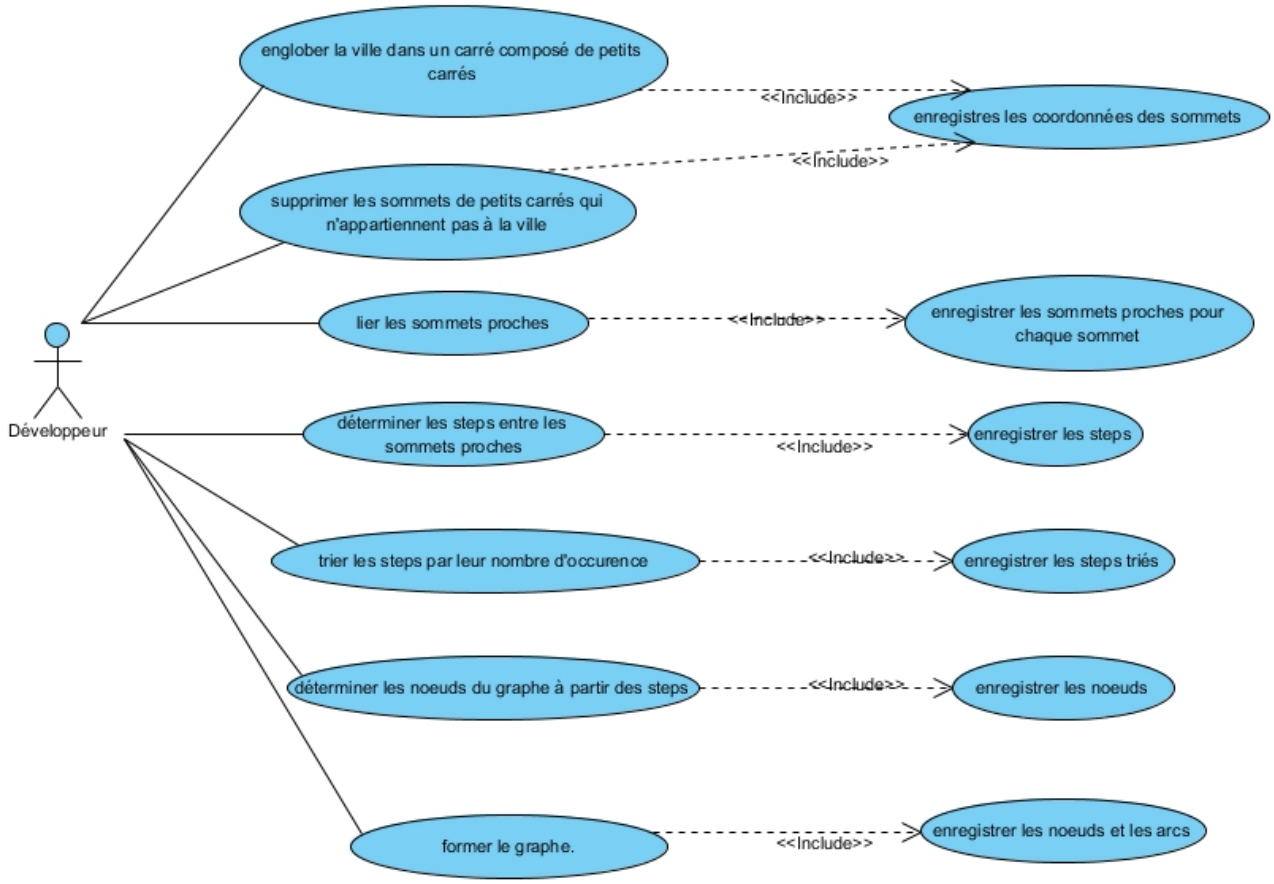


FIGURE 3.2 – Diagramme des cas d'utilisations de génération d'un graphe automatique

3.5.3 Prédiction du trafic

Pour la prédiction du trafic routier, notre application doit générer une représentation idéalisée de la réalité à l'administrateur du web-service. En effet il peut :

- Récupérer l'historique du trafic routier sous forme de séries temporelles. Il a le choix de choisir l'espacement temporel des séries et le nombre de jours suffisant pour générer le bon modèle statistique. Chaque série temporelle est caractérisée par l'id du nœud du départ et d'arrivée d'une route.
- Entraîner les données du trafic routier. Cette phase consiste à générer une modèle autorégressif pour une partie de données de la série puis faire la prédiction et comparer les valeurs obtenues aux valeurs réelles et nous refaisons le même traitement plusieurs fois pour obtenir un modèle très proche de la réalité.
- Consulter les statistiques d'entraînement. Les statistiques concernent les erreurs de prédiction pour chaque modèle générée pendant l'entraînement.
- Prédire le trafic routier. Le principe de prédiction consiste à charger le modèle de prédiction et récupérer les valeurs passée demandés par ce modèle pour exécuter la prédiction.

La figure 3.3 montre le diagramme des cas d'utilisation détaillé pour la partie prédiction du trafic.

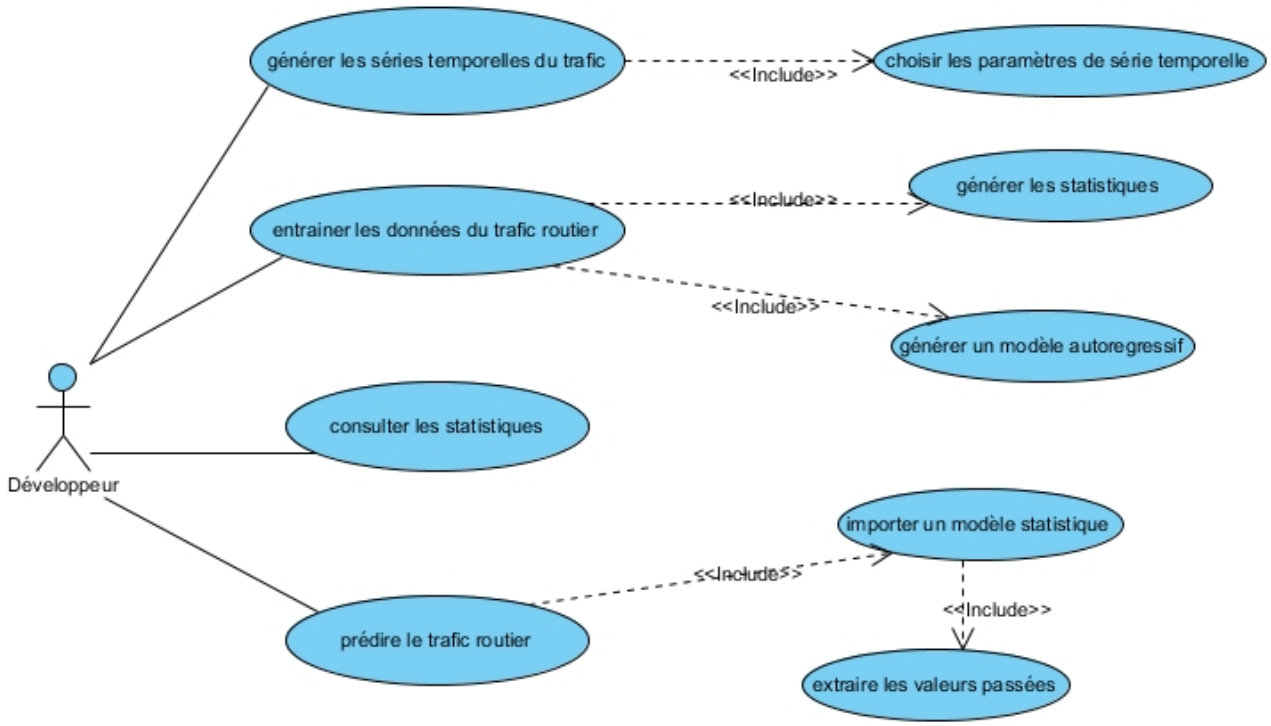


FIGURE 3.3 – Diagramme des cas d'utilisations de prédiction du trafic routier

3.5.4 Calcul du temps de parcours pour un chemin optimal

L'application doit permettre à l'internaute de trouver entre deux points géographiques une route qui donne la plus courte durée de trafic. Ainsi il peut :

- paramétrer le graphe. les coûts des arcs sont chargés à l'instant du choix du temps car les coûts représentent un trafic durant une durée précise.
- déterminer le plus court temps de parcours entre deux points géographiques. L'internaute donc reçoit le temps de parcours et le chemin à parcourir .

La figure 3.4 montre le diagramme des cas d'utilisation détaillé pour la partie calcul du temps de parcours.

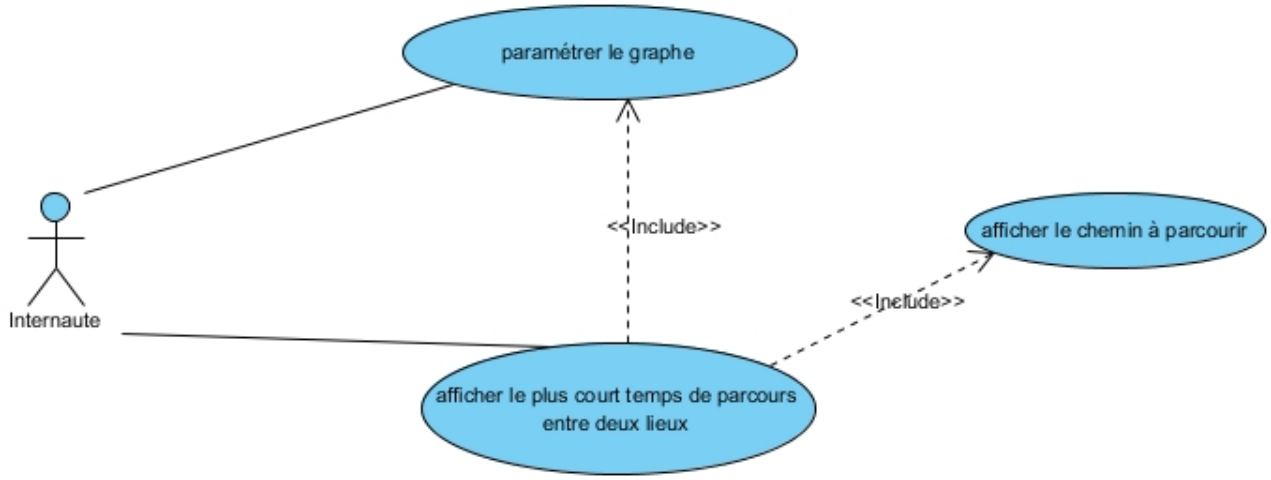


FIGURE 3.4 – Diagramme des cas d'utilisations du calcul du temps de parcours

Conclusion

Au cours de cette phase d'analyse, nous nous sommes penchés sur l'étude de l'application qui nous a permis d'identifier les besoins fonctionnels et de les représenter formellement par les diagrammes de cas d'utilisation. Tout ce travail a été élaboré afin de faciliter le passage à la phase suivante, dans laquelle nous procéderons à la conception détaillée de notre application.

Chapitre 4

Conception

Introduction

Le présent chapitre consiste à présenter l'aspect conceptuel de la solution à développer, une étape primordiale avant l'implémentation. Nous allons définir l'architecture du système que nous allons adopter et détailler les schémas conceptuels des différents modules à implémenter.

4.1 Conception globale

Nous présentons, dans cette section, l'architecture globale de la solution que nous proposons. Cette première phase de la conception consiste à scinder les tâches de l'application en différentes parties ce qui nous permet d'expliciter les fonctionnalités requises pour notre système et de souligner la répartition modulaire de notre solution tout en mettant l'accent sur les différents composants.

4.1.1 Architecture générale

Notre projet se base essentiellement sur trois grandes parties. La première partie touche à la préparation des données du structure du graphe d'une ville pour la bonne représentation des points importants des utilisateurs des routes. La deuxième partie concerne l'entraînement des données du trafic pour prédire des durées du parcours des chemins au futur. Enfin, la partie de cheminement qui concerne le calcul du plus court temps du parcours d'un chemin. Nous justifions, dans le paragraphe suivant, le choix de l'architecture de la solution.

La figure 4.1 nous donne une vue globale de notre application à travers la spécification des différents modules qui la composent.
Notre but est d'assurer un service pour les taximan et les clients de taxi qui retourne le meilleur chemin assurant le plus court temps de parcours.

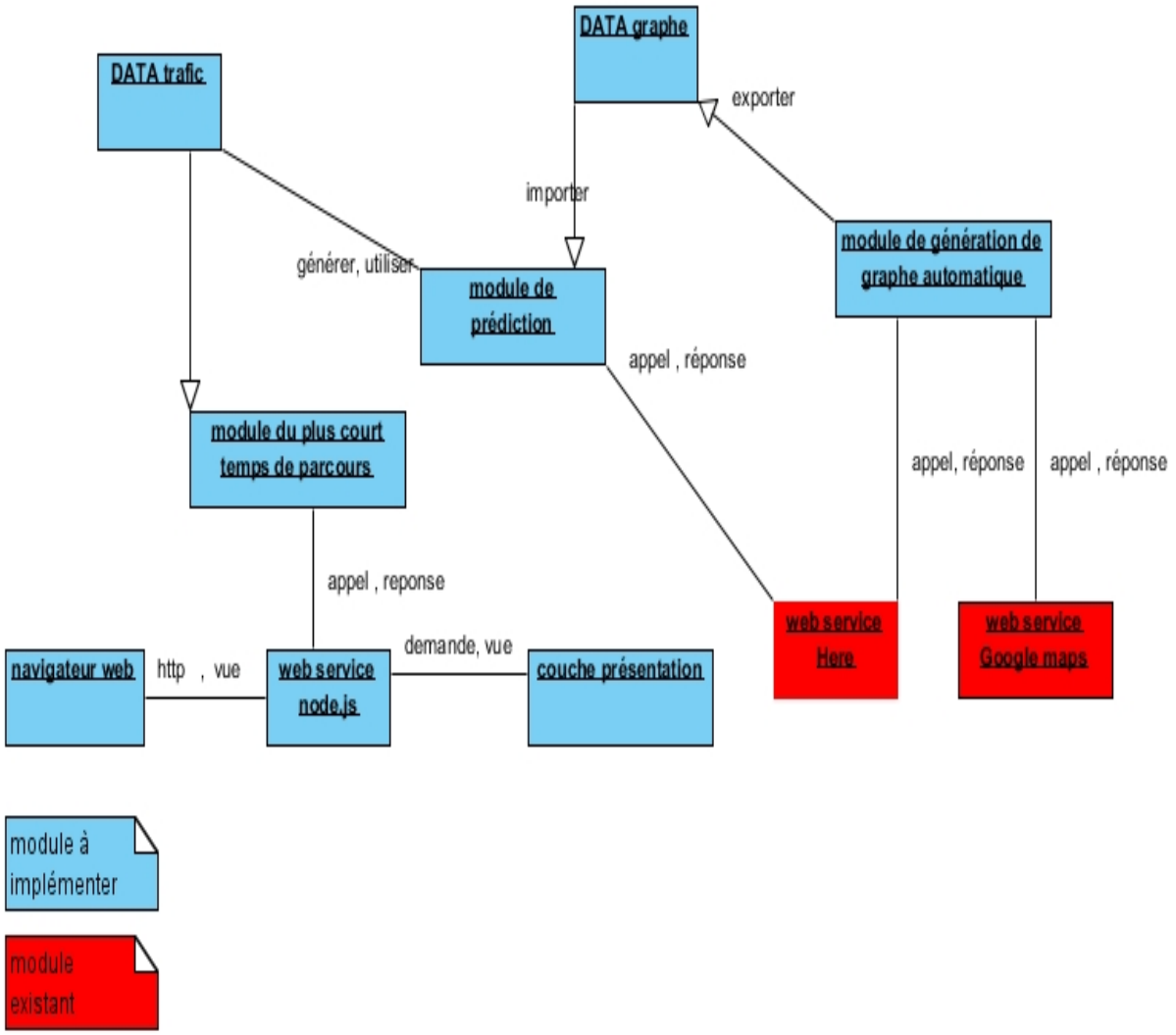


FIGURE 4.1 – Architecture générale de l'application.

4.1.2 Utilisation des services web Nokia Here et Google maps

Un des buts de notre conception de ce projet est le remplacement de ces services. Après une étude approfondie de webmapping, nous avons choisi ces deux qui répondent à nos problèmes. Les données offertes par Google maps et nokia Here permet de nous aider à établir un bon schéma routier qui répond à nos exigence de précision. le module de création d'un graphe automatique fait des appels à ces services pour identifier les tronçons importants des routes. Le module de prédiction aussi fait appel à ses services pour former une base de données locale. Enfin, notre application se libère de ces services et toutes ses ressources deviennent locales.

4.1.3 L'architecture de l'environnement NodeJS

Après une vue d'ensemble sur l'écosystème de NodeJS, nous pouvons dire que son architecture n'est ni exhaustive ni définitive car il est un domaine en constante évolution.

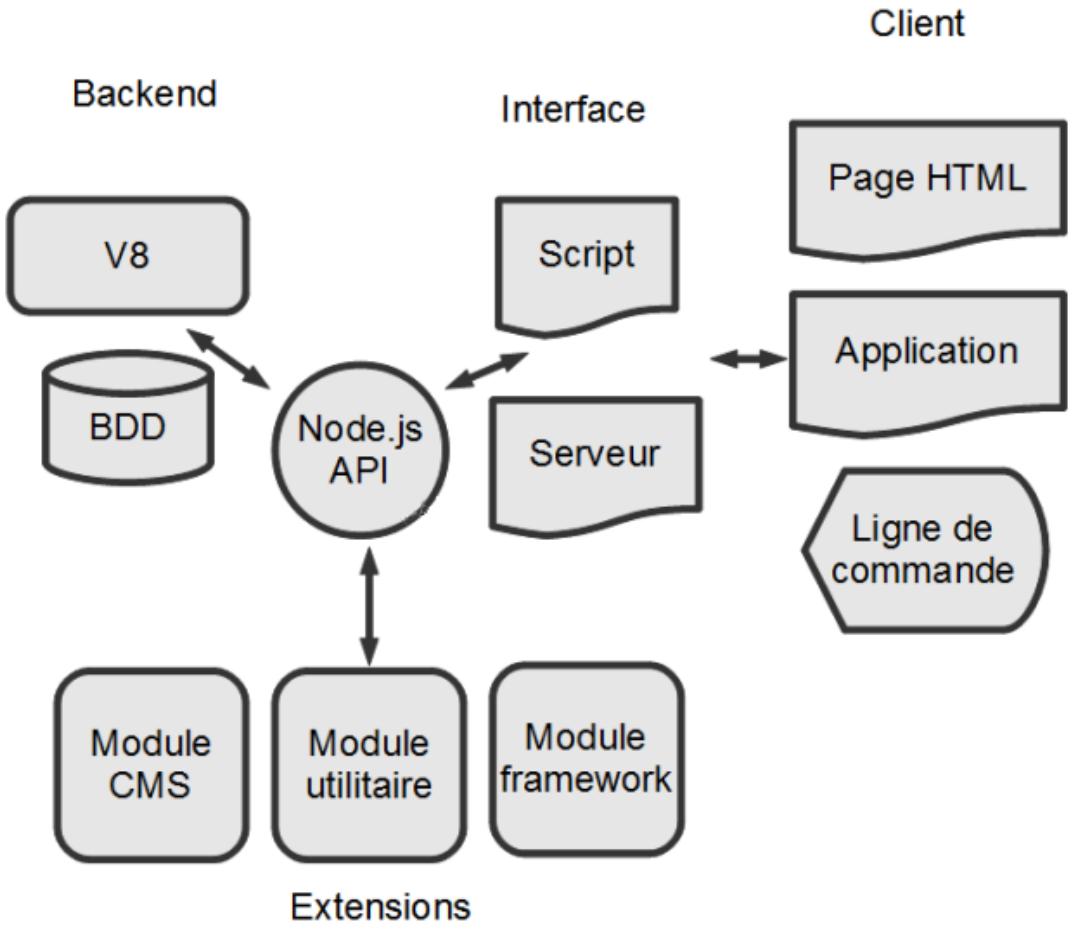


FIGURE 4.2 – Architecture de l'environnement NodeJS

La figure 4.2 montre qu'il existe quatre composants de base qui sont la partie Backend, les extensions, l'interface (avec le client et le serveur) et le client lui même.

Le développeur a un certain niveau de liberté pour modifier cette structure mais tout en gardant les composants de base. Nous allons retourner pour définir chaque composant :

- La partie Backend :Cette partie est basée spécialement sur la machine virtuelle V8 développée par Google qui implémente les spécifications CommonJS.
- Le serveur :NodeJS n'est pas vraiment un serveur, mais un framework pour atteindre un serveur d'application par un script utilisant l'API qu'il fournit.
- Les extensions :Parmi les points forts de NodeJS, c'est le grand nombre et la diversité des modules qui peuvent être utilisés. Ces extensions sont sous forme de packages manipulées par NPM (Node Package Manager). Nous trouvons deux types d'extensions qui fonctionnent soit côté serveur ou côté client.
- La partie client :NodeJS est connu par son serveur Web, on commence par un

script sur le serveur mais c'est réduit car les modules seront des fichiers qui s'exécutent dans la partie client.

Nous pouvons toujours utiliser du JavaScript côté client pour manipuler la page HTML. La nouveauté que NodeJS offre un environnement côté serveur qui nous permet aussi d'utiliser le langage JavaScript pour générer des pages web. En gros, il vient en remplacement de langages serveur comme PHP, Java EE, etc. La figure 4.3 illustre un scénario entre le client et le serveur NodeJS

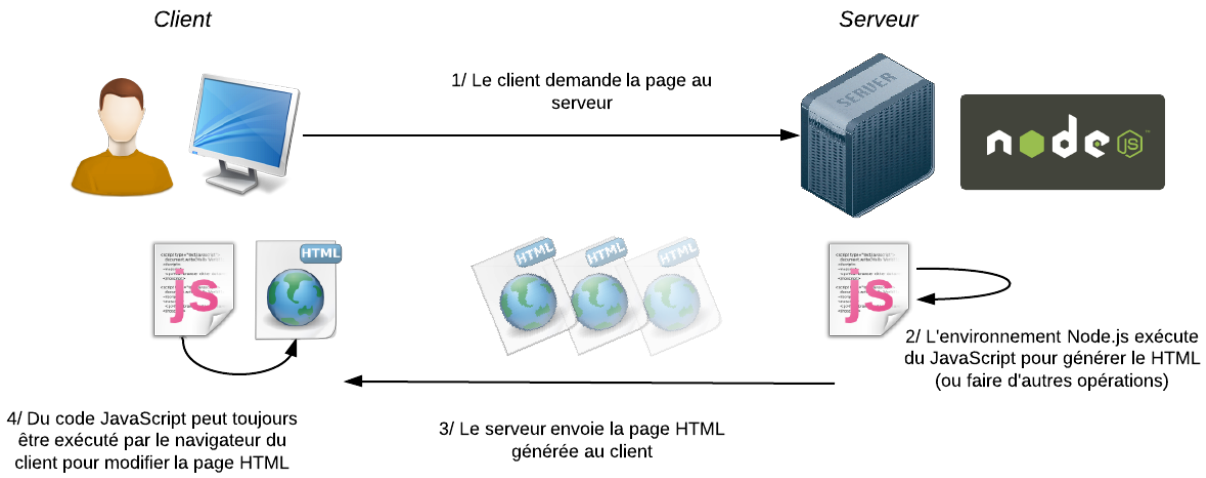


FIGURE 4.3 – Architecture d'un serveur NodeJS

4.2 Conception détaillée

Une fois la phase de conception globale est achevée, nous exposons dans cette partie la phase de conception détaillée dans laquelle nous développons davantage les modules et les composants de notre solution précédemment décrits d'une manière générale.

4.2.1 Etude de module de plus court temps de parcours d'un chemin

Ce module prend comme input deux fichiers et des données envoyées par le serveur NodeJS. Le premier est celui qui décrit les sommets et les arcs et les caractéristiques. Le deuxième comporte les données du trafic d'un jour bien déterminé. Les formats de ces fichiers suivent le format CSV. Pour que ce module s'exécute, il a besoin de la date et de l'heure et une origine et une destination.
La figure 4.4 présente le diagramme de module plus court temps de parcours d'un chemin. Elle illustre l'exécution de ce module en huit étape.
Etape 1,2,3 : collecter des paramètres de l'exécution de l'algorithme.
Etape 4 : appeler le sous module construction du graphe par le sous module algorithme A étoile .

Etape 5 : importer un graphe générique.
Etape 6 : affecter des durées de parcours entre les nœuds pour chaque arc.
Etape 7 : exécuter l'algorithme A*.
Etape 8 : retourner le temps de parcours le plus court par l'algorithme A* entre l'origine et la destination avec les nœuds parcourus.

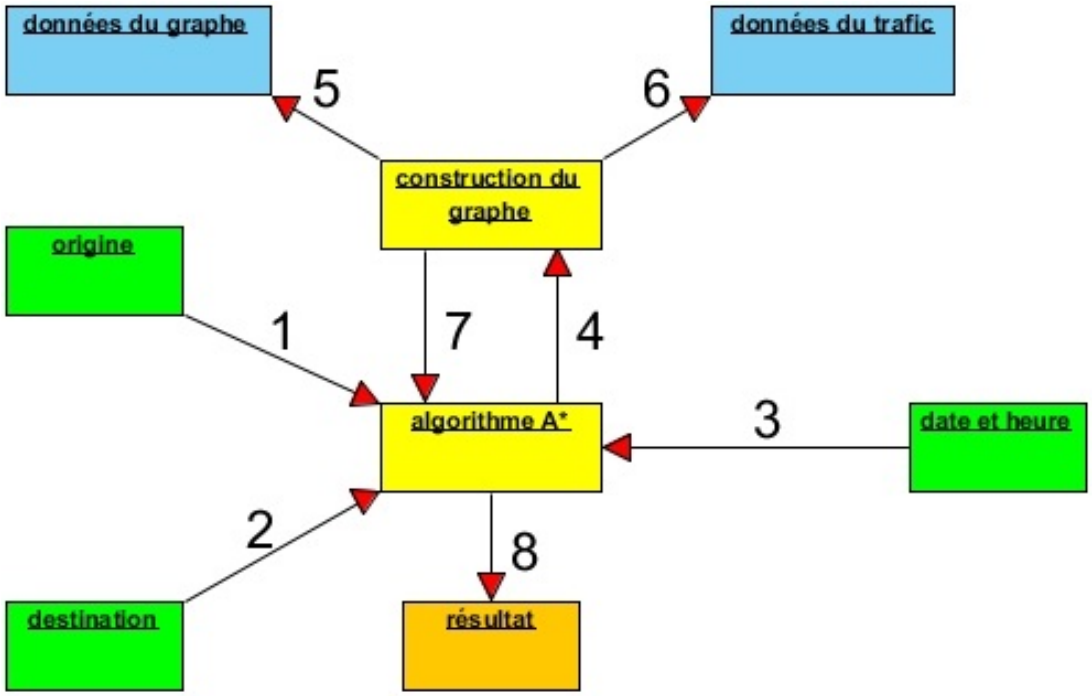


FIGURE 4.4 – Architecture de module de plus court temps de parcours

4.2.1.1 Diagramme de classe

La figure 4.5 présente le diagramme de classe de module plus court temps de parcours. La classe Graphexection hérite de la classe GraphValue. La classe GraphValue assure la phase d'initialisation et de construction du graphe en important des données sauvegardées.les attributs de cette classes sont :
1.g : représente le graphe . Son type est une structure d'un graphe.
2.nbrSommet : représente le nombre de sommet du graphe.
3.num_vertex : chaque arc a un identifiant enregistré dans num_vertex.
4.valeur : représente le coût d'un arc.
5.mapcost :il enregistre les différentes coûts des arcs pour différentes date et heure.

Les méthodes de cette classes sont :
1.changecost : elle assure le changement des coûts des arcs pour affecter les valeurs correspondantes au temps voulu.
2.init : elle initialise la connexion entre la classe et le serveur du NodeJS .

3.New : elle assure l'instanciation de cette classe dans le serveur NodeJS.

La classe Graphexecution assure le déroulement de l'algorithme A* pour retourner finalement les résultats demandés. Elle comporte les méthodes suivantes :
1.costchange : elle fait appel à changecost à partir du serveur.
2.astarexec : elle fait appel à Affastar à partir du serveur.
3.Affastar : elle exécute l'algorithme A*.

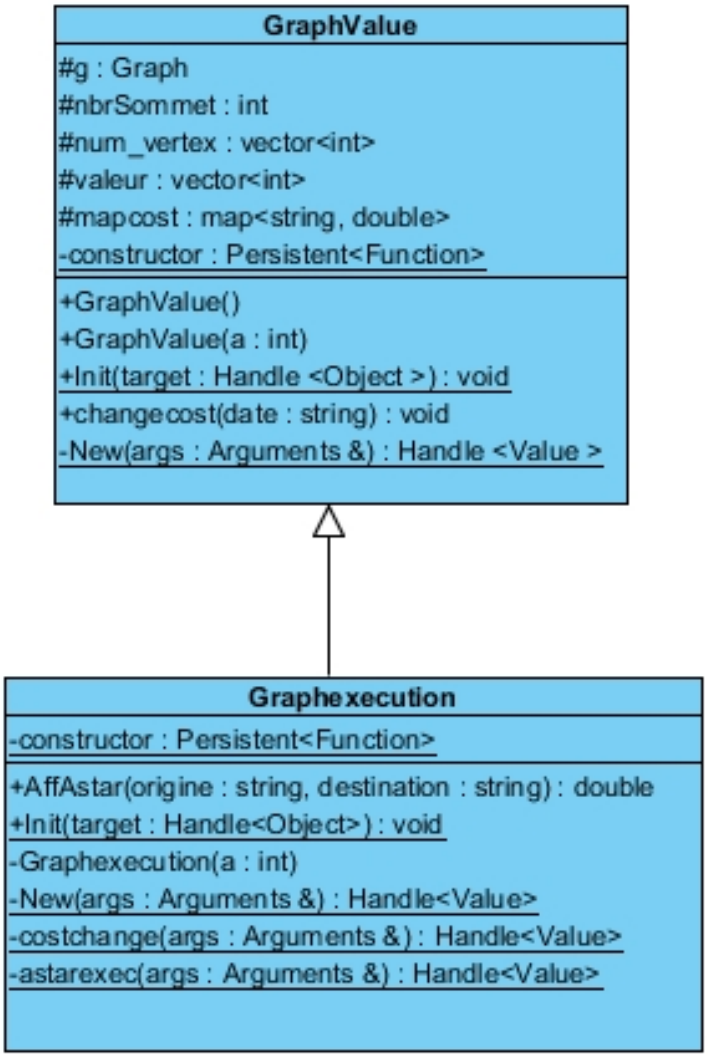


FIGURE 4.5 – Diagramme de classe de plus court temps de parcours

4.2.2 Etude de module de génération de graphe automatique

Ce module comporte plusieurs sous module pour la synthèse d'un graphe routier optimal d'une ville. La figure 4.6 illustre les différentes étapes pour générer un graphe optimal. Ce module comporte huit parties importantes.
partie 1 : déterminer les quatre points extrême qui limite la ville pour former un carré.

partie 2 : suivant les paramètres entrés de longueur de coté de carré diviser la grande carrés en des petit carrés. partie 3 : faire des appel au service web Google maps pour éliminer les points des carrés qui n'appartiennet à la ville.
partie 4 : pour chaque point calculer les durées du parcours en appelant le service web Nokia Here pour atteindre les autres points et choisir les dix premiers pour créer des arc entre eux.
partie 5 : en faisant appel au service web Here nous importons les routes de chaque arc. Les routes contiennent plusieurs propriétés et caractéristiques.
partie 6 : extraire les tronçons des routes et les enregistrer dans un fichier.
partie 7 : calculer le nombre d'occurrences de chaque tronçon dans ce fichier. Le développeur fixe le nombre d'occurrences pour qu'un tronçon ne soit pas rejeté.
partie 8 : nous répétons la partie quatre sur les points retenus. Le graphe obtenu sera stocké dans un fichier XML.

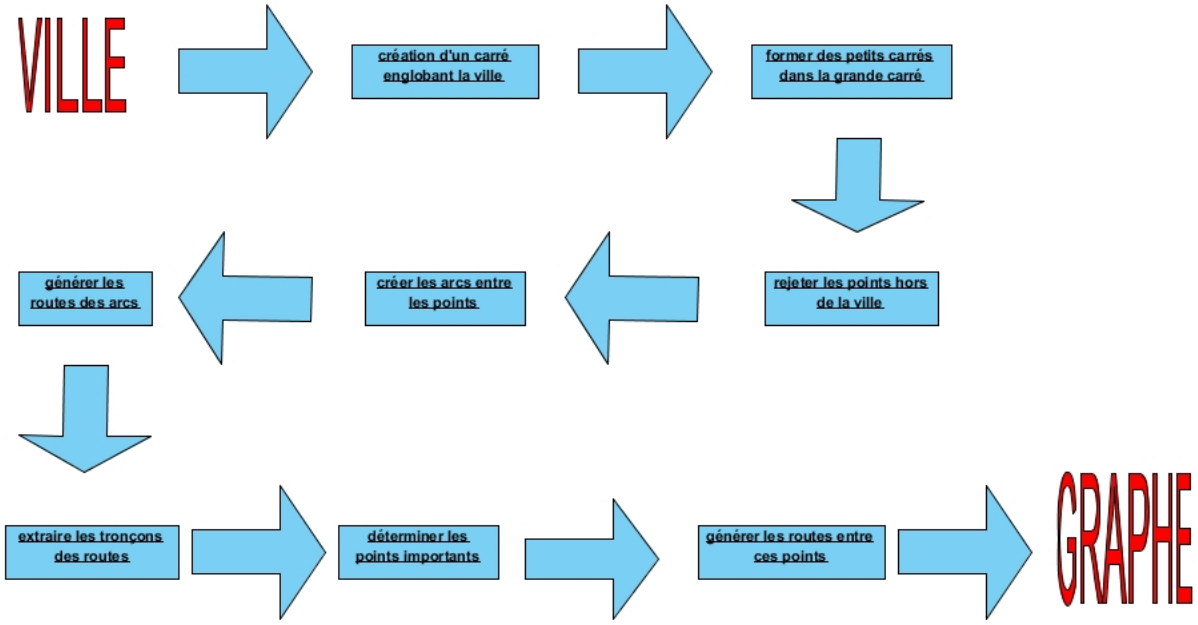


FIGURE 4.6 – Architecture de module de génération de graphe automatique

4.2.2.1 Diagramme de séquence

La figure 4.7 du diagramme de séquence de génération d'un graphe montre la chronologie des appels entre le développeur et les trois modules. A chaque fois le développeur doit attendre le retour d'appel pour passer le prochain appel.

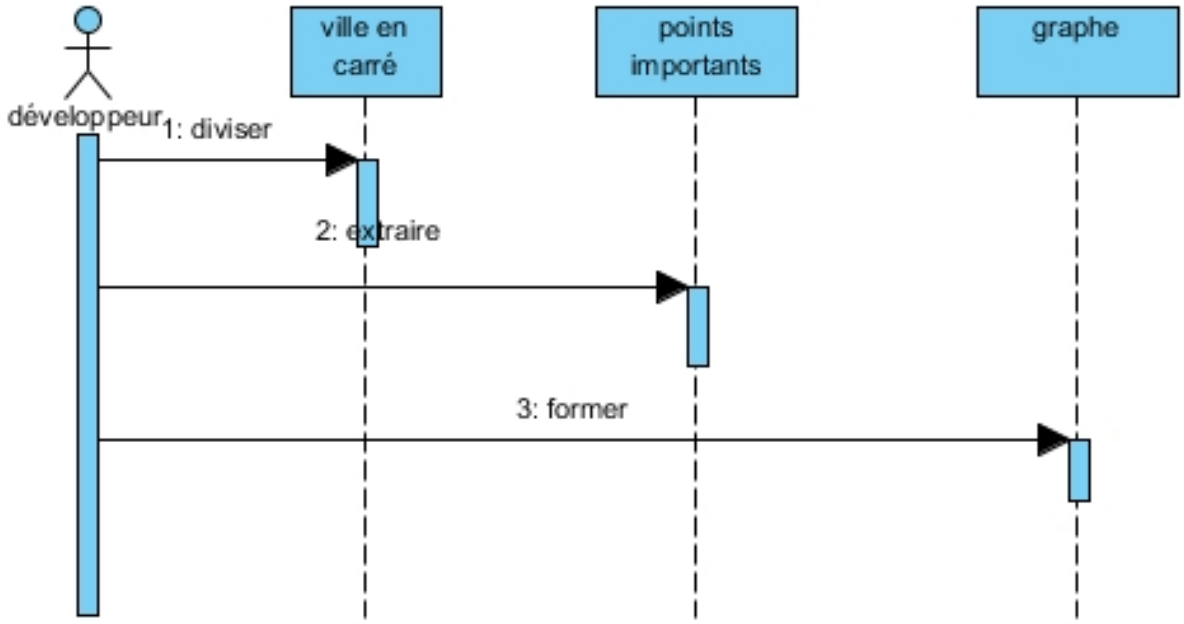


FIGURE 4.7 – Diagramme de séquence de génération d'un graphe

4.2.3 Etude de module de prédiction

La figure 4.8 illustre la conception de module de prédiction. Trois fonctions principales doit être réalisées.
La première fonction : concerne la génération des séries temporelles du trafic des routes. nous appelons le web service Nokia Here pour recevoir pour chaque route une série temporelle. chaque série temporelle doit indiquer le nœud d'origine et celui de destination et la date et l'heure et la durée. Elles sont stockées dans des fichiers.
La deuxième fonction : concerne l'apprentissage (l'entraînement) des séries temporelles pour aboutir à générer un modèle autoregressif tout en faisant appel aux théorèmes mathématiques. Chaque modèle a ses caractéristiques qui sont générés dans les statistiques de ce modèle.
la troisième fonction : elle prend en entrée le modèle généré après l'apprentissage des séries temporelles. Suivant le modèle qui demande des valeurs passées dans les séries temporelles elle calcule la valeur de prédiction. Elle permet aussi de générer des statistiques en comparant les valeurs prédites avec les valeurs réelles pour valider le modèle.

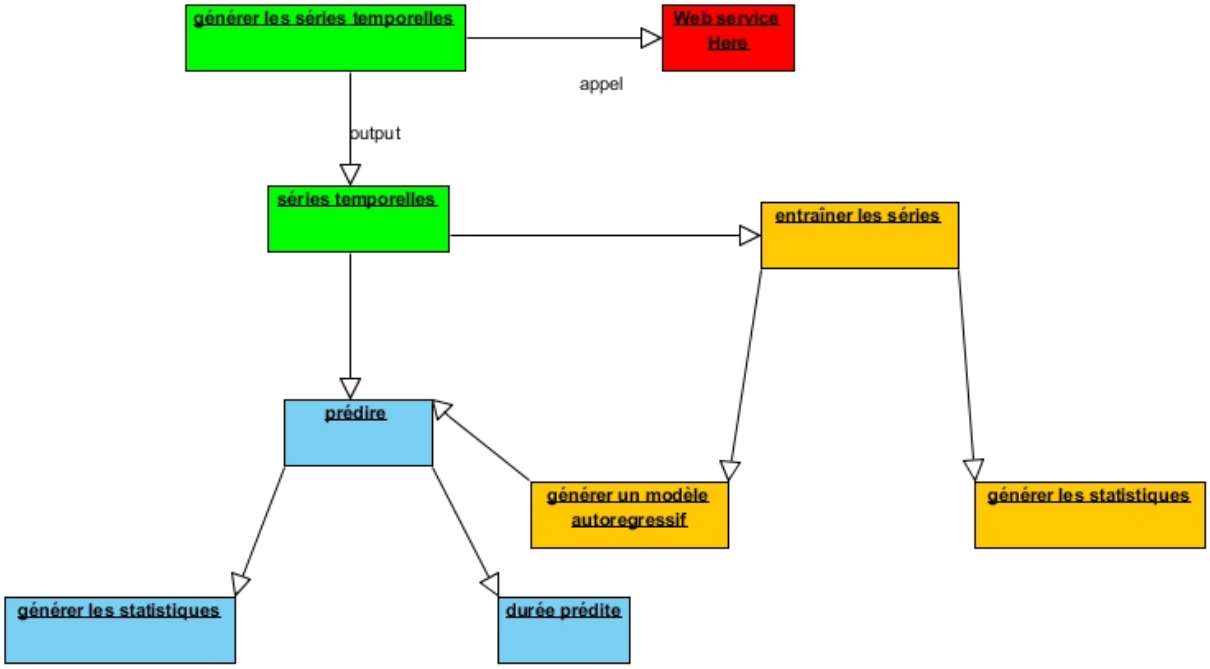


FIGURE 4.8 – Architecture de module de prédiction

4.2.3.1 Diagramme de séquence

Dans ce qui suit nous allons présenter le diagramme de séquence dans la figure 4.8. L'administrateur du système doit attendre la finalisation de la tâche de génération des séries temporelles pour entamer l'apprentissage. Une fois l'apprentissage génère le modèle de prédiction autoregressif l'administrateur peut prédire les valeurs des durées en trafic au futur.

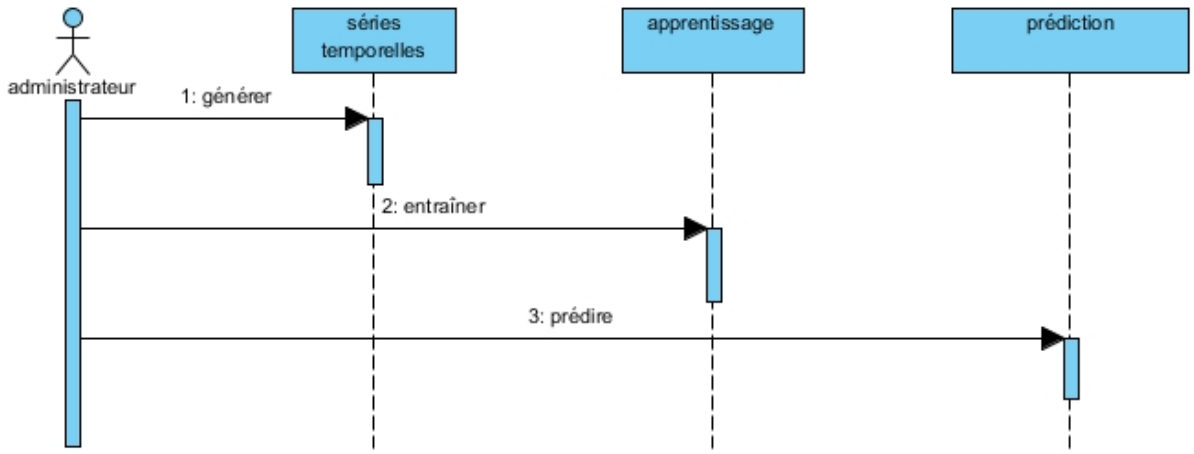


FIGURE 4.9 – Diagramme de séquence de prédiction

Conclusion

Ce chapitre a dévoilé la conception qui est une phase fondamentale dont la réussite d'un projet en dépend. La conception de notre projet étant ainsi achevée nous passons au dernier volet de notre rapport qui est la phase de réalisation. Le dernier chapitre est consacré à la présentation des technologies utilisées au cours de développement et l'exposition des quelques imprimes écrans de notre application.

Chapitre 5

Réalisation

Introduction

Après la description détaillée et l'étude approfondie des différentes phases du projet qui nous a permis d'appréhender le fonctionnement du système, organiser l'ensemble des besoins et concevoir des solutions, nous nous intéressons à présent aux outils qui permettent de mettre en place notre système. En effet, ce chapitre représente le dernier volet de ce rapport et il a pour objectif d'exposer l'application réalisée. Nous présentons en premier lieu l'environnement de travail sous ses deux aspects : matériel et logiciel. Nous abordons aussi les différentes technologies et outils de développement utilisés. En deuxième lieu, nous illustrons la réalisation de notre travail en exposant quelques aspects de l'application à travers des captures écrans.

5.1 Environnement de travail

Nous présentons dans cette section les choix techniques relatifs à l'environnement matériel et logiciel qui ont contribué à la réalisation du notre service de voiture de tourisme avec chauffeur.

5.1.1 Environnement matériel

Pour la réalisation de ce projet, nous nous sommes munis d'un poste de travail et un ordinateur portable disposant d'une connexion internet ayant les caractéristiques illustrées par le tableau 5.1.

TABLE 5.1 – Environnement matériel utilisé.

	Processeur	**SE**	**RAM**	**Disque dur**
Poste de travail	CPU G630@2.70GHZ	Windows XP 32 bits	4 GO	250 GO
PC	IntelőCore i5	Ubuntu 14.04 64 bits	8 GO	600 GO

5.1.2 Environnement Logiciel

Afin de concrétiser la solution conçue et la mettre en place, nous nous sommes servis de l'environnement logiciel suivant :

- Environnement du développement : visual c++ express ;
- Editeur de texte : Notepad++ , vim ;
- Outil de débogage : Firebug ;
- Modélisation UML : Visual Paradigm.

5.1.3 Choix des langages

Nous consacrons cette section pour justifier nos choix des langages adoptés pour la mise en place du notre application.Le module de génération du graphe automatique est développé en Python et l'affichage des cartes se fait avec le langage JavaScript. Les autres modules est développées en C++. Le serveur web est développé en JavaScript et HTML5.

Choix du C++

C++ est non seulement un langage de programmation puissant conçu pour être sur inter plate-forme et international, mais aussi un environnement de développement qui est continuellement étendu pour fournir de nouvelles caractéristiques et bibliothèques permettant de gérer de manière élégante des problèmes traditionnellement complexes dans les langages de programmation classiques. Du point de vue du programmeur, C++ permet de réduire le temps de développement dune application grâce à la réutilisation du code développé. Ce sont les raisons pour lesquelles nous avons choisi ce langage.

Choix du Python

Python est un langage portable, dynamique, extensible, gratuit, qui permet sans l'imposer une approche modulaire et orientée objet de la programmation. Python est développé depuis 1989 en logiciel libre par Guido van Rossum et de nombreux informaticiens bénévoles. Sa puissance de calcul est beaucoup mieux que C++. Il est très adapté pour développer des outils de calcul rapidement. Ses principale caractéristiques sont :

- Il est portable, non seulement sur les différentes variantes d'unix, mais aussi sur les OS propriétaires : MacOS, BeOS, NeXTStep, MS-DOS .. et les différentes variantes de Windows.
- Il est gratuit, mais on peut l'utiliser sans restriction dans des projets commerciaux.
- Il convient aussi bien à des scripts d'une dizaine de lignes de code qu'à des projets complexes de plusieurs dizaines de milliers de lignes.

- Il est aussi multi-threadé.
- Il intègre, comme Java ou les versions récentes de C++, un système d'exceptions, qui permettent de simplifier considérablement la gestion des erreurs.

5.1.4 Choix technologiques

Nous consacrons cette section pour justifier nos choix technologiques adoptés pour la mise en place du notre application.

Plateform NodeJS

Node.js est une plate-forme construite sur l'exécution JavaScript de Chrome pour construire facilement des applications de réseau rapides et évolutives. Node.js utilise un modèle non bloquant I/O (Input/Output=Entrée/Sortie) événementielle qui le rend léger et efficace, idéal pour les applications en temps réel qui fonctionnent sur des différents dispositifs. Nous justifions notre choix par :

- Licence Open Source , sous licence MIT
- Projet très actif et soutenu par de grands comptes (Microsoft, Google)
- Un seul langage Frontend et Backend
- Il offre une documentation riche ;
- Gestionnaire de modules performant et très simple
- Simplicité de mise en œuvre
- Mode Mono Thread basé sur la gestion dévénements
- De tres nombreux modules disponibles

Bibliothèque Gretl

La phase dapprentissage et de prédiction autorégressive nécessite des API spécialiseés dans le domaine statistique. Nous avons choisis la bibliothèque Gretl qui répond à nos besoins. Elle a une structure spécifique pour les données ce qui permet de réduire lespace mémoire utilisé. Elle permet aussi de tester la stationnarité. Cette bibliothèque retourne des résultats précis après lapprentissage et la prédiction. Elle est aussi multiplateforme ce qui répond au besoin de portabilité.

Bibliothèque Boost Graph

Boost Graph (BGL) propose une interface standard pour manipuler des graphes. Cette interface est similaire aux conteneurs de la bibliothèque standard et permet l'accès aux différents éléments à l'aide d'itérateurs. Il est donc possible d'utiliser les algorithmes de la bibliothèque standard pour travailler sur les graphes ou d'utiliser les algorithmes

spécifiques pour les graphes fournis par Boost Graph. On peut aussi manipuler les éléments du graphe, accessibles via la classe de traits : les sommets (ou nuds, vertex dans BGL), les arcs (pour un graphe orienté ou arêtes pour un graphe non orienté, edge dans BGL) et le graphe lui-même (graph dans BGL).

5.2 Implémentation

Notre application de réservation de transport de personnes permet davoir pour un trajet donné et une date future donnée une estimation du temps de parcours avec trafic. Ceci est assuré par une structuration bien organisée des différentes fonctionnalités offertes par l'ensemble des modules conçus et implémentés durant notre stage.

5.2.1 Génération d'un graphe automatique

Nous avons mis en place un outil de génération d'un graphe automatique pour une ville donnée. Notre projet s'intéresse à la ville de Paris. La première étape pour la réalisation de cet outil consiste à englober la ville dans un grande carré contient des petits carrés. La figure 5.1 montre la ville de Paris après un traitement de détermination des limites de la ville pour entamer les calculs des coordonnées de chaque point dessiné dans la carte.

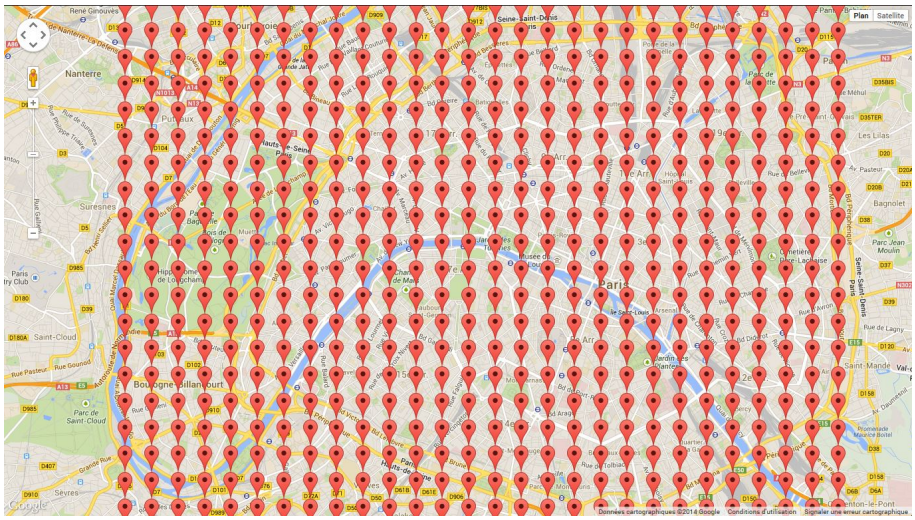


FIGURE 5.1 – Ville de Paris englobée dans un grand carré

L'étape suivante consiste à supprimer les points en dehors de Paris. Pour la réalisation de cette étape, nous appelons le service web de Google maps pour déterminer les

frontières internes de la ville de Paris. Donc, chaque point va passer par le test d'appartenance à Paris. Le résultat est illustré par la figure 5.2 montre que tous les points qui n'appartiennent pas à Paris sont supprimés.

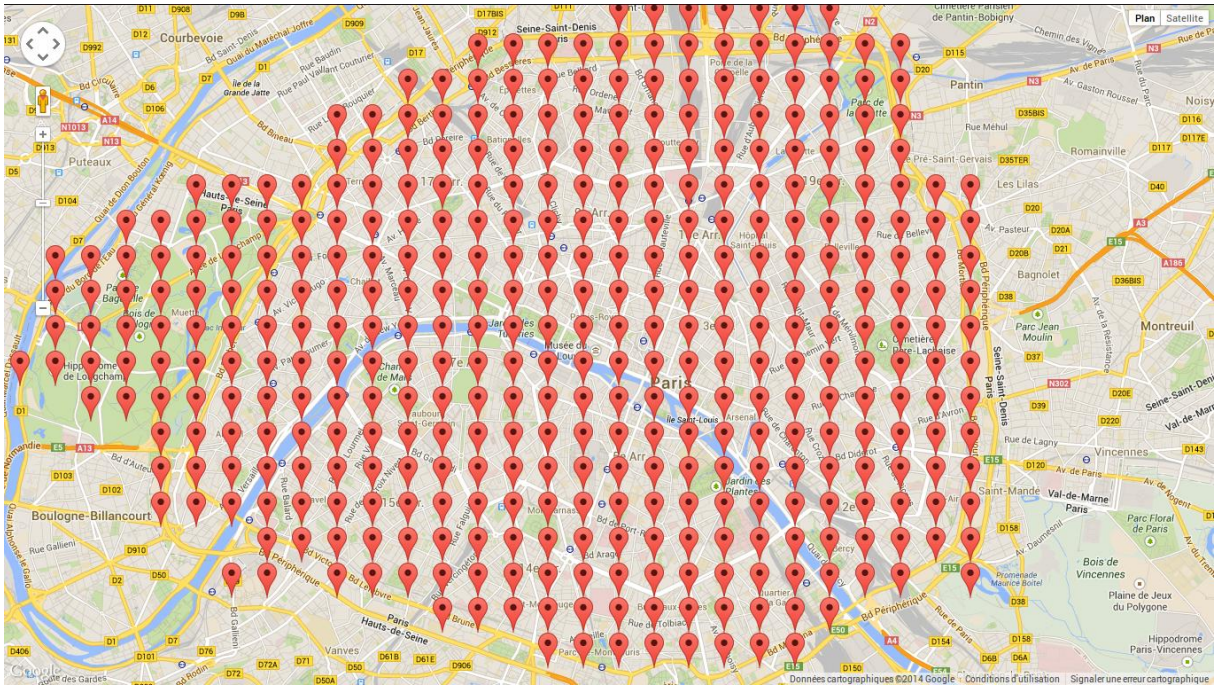


FIGURE 5.2 – Ville de Paris divisée en carré

Nous passons maintenant à la création des arcs entre les points obtenus. Alors, notre entrée est les coordonnées géographiques des points dans la ville de Paris et la sortie est un fichier décrivant les arcs entre ces nœuds(points). Pour chaque nœuds nous calculons la distance qui le sépare des autres nœuds puis nous trions les distances par ordre décroissant. Les premiers nœuds qui se trouvent dans un cercle de rayon paramétrable vont être retenus pour créer les arcs entre le nœud traité et les nœuds retenus. La figure 5.3 représente un fichier contenant les arcs qui relient les nœuds entre eux en utilisant un id pour chaque nœud.

```
N/A:origine:destination
A:1:2
A:1:3
A:1:7
A:1:6
A:1:8
A:1:4
A:1:9
A:1:12
A:1:11
A:1:13
A:1:5
A:1:14
A:1:10
A:1:21
A:1:22
A:1:20
A:1:15
A:2:1
A:2:7
A:2:3
```

FIGURE 5.3 – Le fichier des arcs générés

Après avoir créé les arcs, nous cherchons une route pour chaque arc qui représente

un couple de nœud. En faisant appel à l'API de Here maps qui répond avec un fichier JSON, nous divisons les routes en des tronçons importants. Chaque tronçon est composé des steps. Ils sont des points importants dans une route. Ils représentent généralement des ronds-points, virage et intersections. Ces steps sont considéré comme les points clé d'une carte routière. Nous extrayons pour chaque route ses steps pour obtenir enfin une base de steps. La figure 5.4 montre les différents steps extraits des routes dans une partie de la ville de Paris.

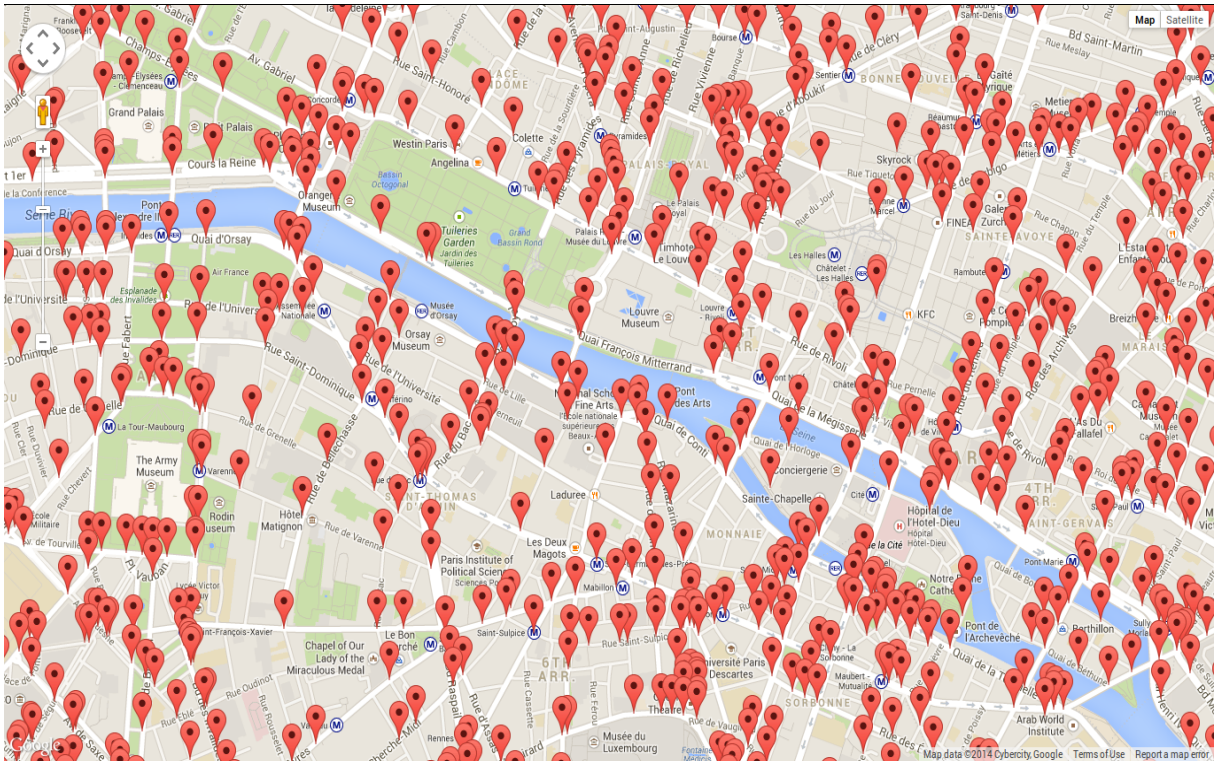
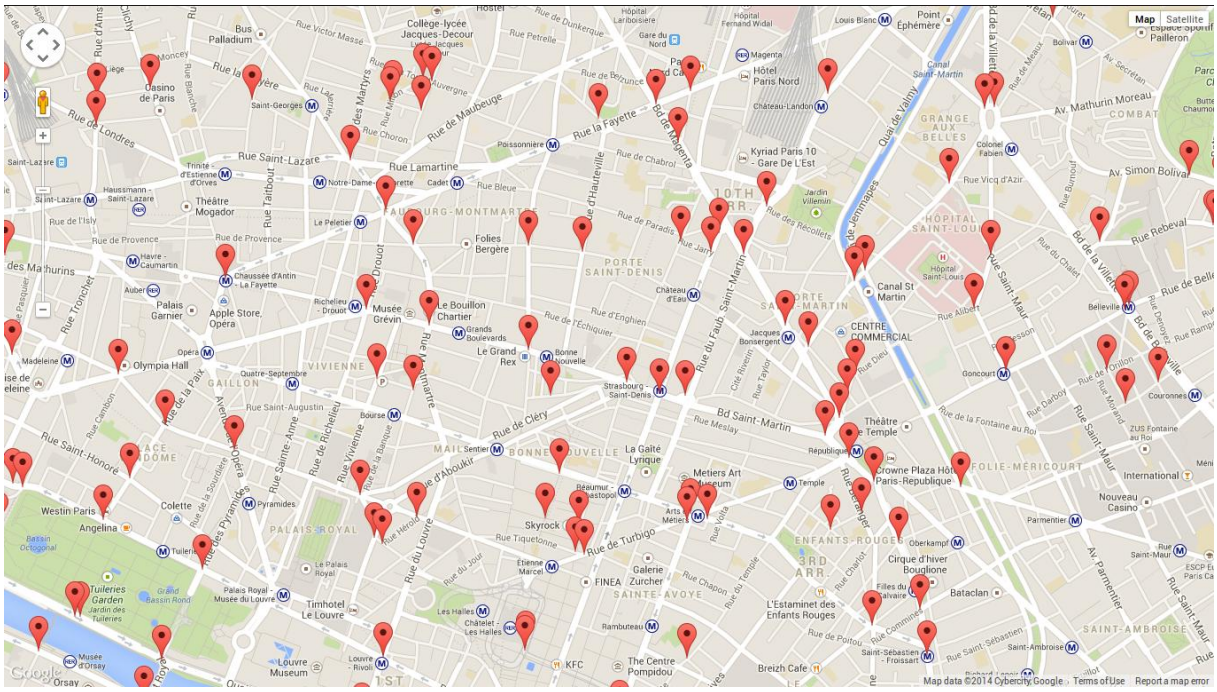


FIGURE 5.4 – La distribution des steps dans la carte

Nous remarquons dans la figure 5.4 un nombre important des steps. Nous n'avons pas besoin de ce grand nombre des steps voisins mais nous avons besoin des plus importants entre eux. Le fichier contenant la base des steps contient un nombre d'occurrences pour chaque step. Nous allons retenir que les steps qui ont un nombre d'occurrences important. Le nombre d'occurrence représente la densité du trafic dans le step. La figure 5.5 représente un fichier généré par un script python pour le calcul d'occurence pour chaque nœud. La figure 5.6 montre les steps retenus. Ils couvrent toute la ville de Paris. Les chauffeurs trouvent que ces points sont inévitables dans la ville. Alors, les nœuds sont détéminé. Nous passons à la création des arcs qui demande le calcul du temps de parcours entre les nœuds. Chaque nœud est relié aux dix nœuds qui ont les plus courts temps de parcours. La figure 5.7 montre un sous graphe de la ville de Paris qui représente l'output de notre module de génération de graphe automatique.

```
 994 48.7295303,2.3664899
 994 48.7298889,2.3663499
 994 48.7300911,2.3694201
 994 48.7313194,2.3662701
 994 48.7319794,2.3680301
 491 48.7382317,2.3647001
 526 48.7399902,2.36374
1017 48.7459183,2.36624
 468 48.7674484,2.3435099
 503 48.7697983,2.3455901
 468 48.7729607,2.34536
 481 48.7787018,2.3468101
 491 48.7869682,2.36748
 526 48.790081,2.3689301
  21 48.7946701,2.34532
 465 48.7994995,2.3422699
  21 48.799511,2.3440001
  13 48.8070183,2.336
   8 48.8088303,2.3306601
  13 48.8108597,2.3390601
```

FIGURE 5.5 – Nombre d'occurrences des steps


Map
Satellite
Rue de la Fayette
Rue Saint-Lazare
Rue de Maubeuge
Rue de Londres
Rue Lamartine
Rue du Faub. Saint-Martin
Bd Saint-Martin
Rue de Turbigo
Rue du Louvre
Rue Saint-Honoré
Rue de Cléry
Rue Saint-Maur
Av. Mathurin Moreau
Av. Simon Bolivar
Rue Rebeval
Rue de Belleville
Rue de la Villette
Rue Saint-Antoine
Rue de Rivoli
Rue de Richelieu
Rue Vivienne
Rue Montmartre
Rue du Bac
Rue de Sèvres
Rue de Grenelle
Rue de Vaugirard
Rue de Rennes
Rue de Seine
Rue de Tournon

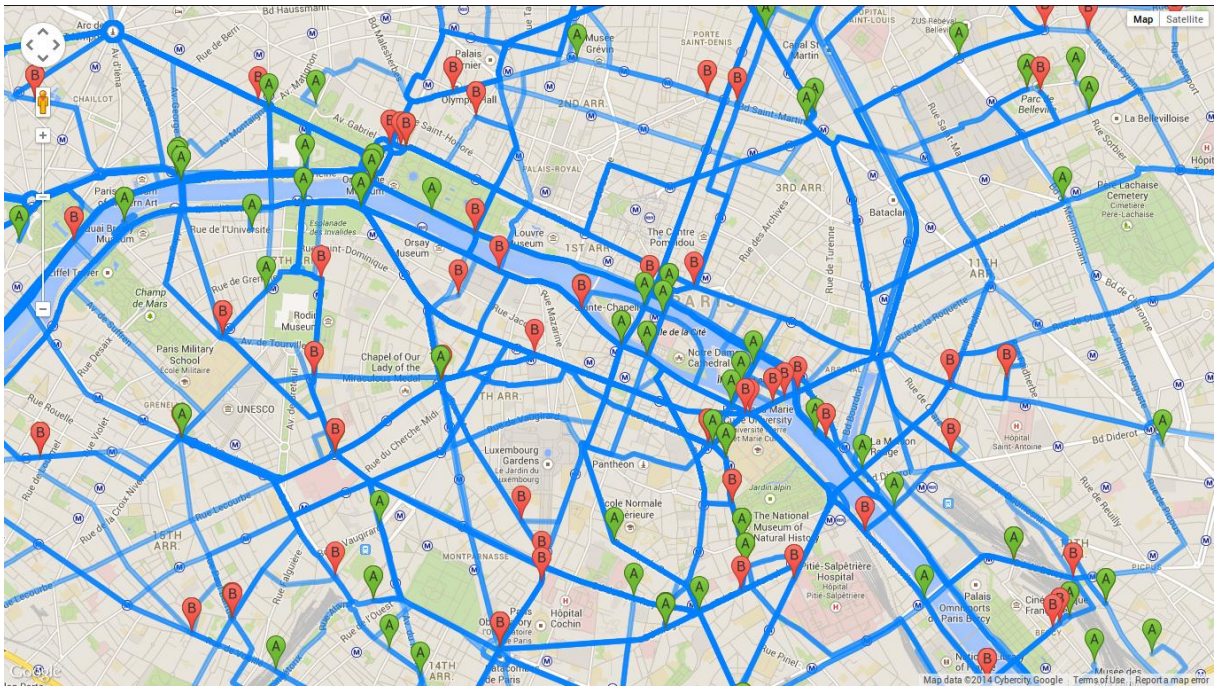


FIGURE 5.7 – Un sous graphe de la ville de Paris

5.2.2 Prédiction du trafic

Notre projet vise à assurer un service de réservation en prenant en compte le trafic routier. Ce service est assuré par un module de prédiction. La réalisation de cette tâche demande une phase de génération des séries temporelles des routes et une phase d'apprentissage et une dernière phase de prédiction. La génération des séries temporelle consiste à enregistrer les valeurs de durée en trafic pour chaque arc du graphe pour chaque dix minutes. La figure 5.8 représente le fichier généré des séries temporelles. Chaque ligne indique la date en début puis l'heure et les minutes et les secondes séparé par deux points puis l'id du nœud d'origine et l'id du nœud de destination et enfin la valeur du temps de parcours.

```
2014-03-03T-00:00:00;1;41;199.0
2014-03-03T-00:10:00;1;41;199.0
2014-03-03T-00:20:00;1;41;199.0
2014-03-03T-00:30:00;1;41;199.0
2014-03-03T-00:40:00;1;41;199.0
2014-03-03T-00:50:00;1;41;199.0
2014-03-03T-01:00:00;1;41;199.0
2014-03-03T-01:10:00;1;41;199.0
2014-03-03T-01:20:00;1;41;199.0
2014-03-03T-01:30:00;1;41;199.0
2014-03-03T-01:40:00;1;41;199.0
2014-03-03T-01:50:00;1;41;199.0
2014-03-03T-02:00:00;1;41;199.0
```

FIGURE 5.8 – Fichier d'une série temporelle

Une fois les séries temporelles sont importées, nous entamons l'étape de l'appren-

tissage. Il prend en entrée une série temporelle d'un arc du graphe. A partir des valeurs passées, l'apprentissage consiste à étudier l'évolution des valeurs pour commencer à mettre en place un modèle. Nous avons choisi le modèle autoregressif. A la fin d'apprentissage un modèle est généré. La figure 5.9 montre un modèle autoregressif à la fin d'apprentissage avec les statistiques en rapport avec le modèle.

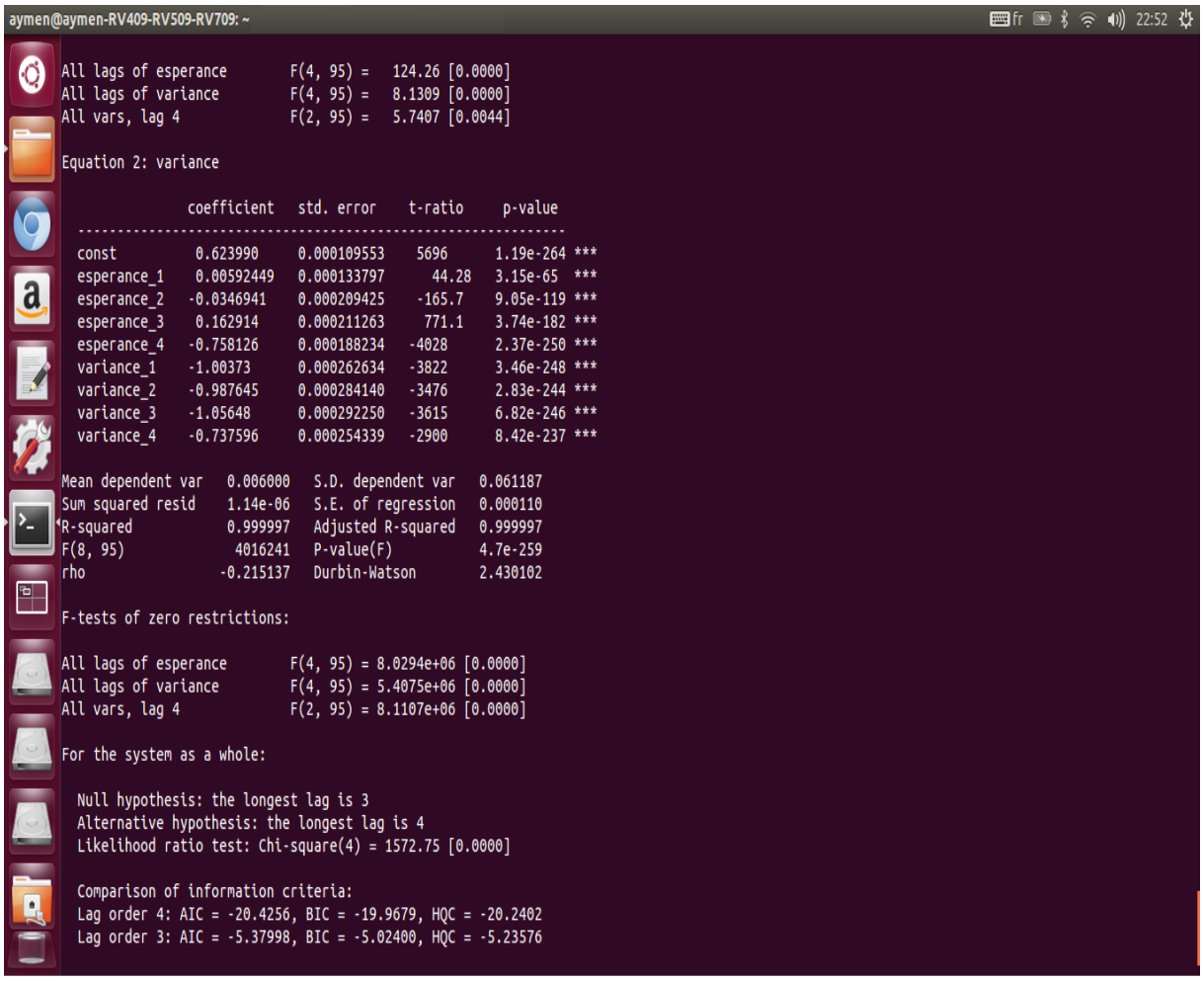


FIGURE 5.9 – L'apprentissage des séries temporelles

Après avoir obtenu le modèle autoregressif, nous passons à la prédiction des valeurs du temps de parcours. Nous avons besoin des valeurs passées demandées par le modèle et nous exécutons la prédiction. Elle consiste à donner une valeur future en se basant au modèle d'apprentissage. La figure 5.10 représente le résultat de la prédiction qui donne des valeurs futures. Les statistiques relatives à la prédiction sont présentes aussi dans cette même figure.

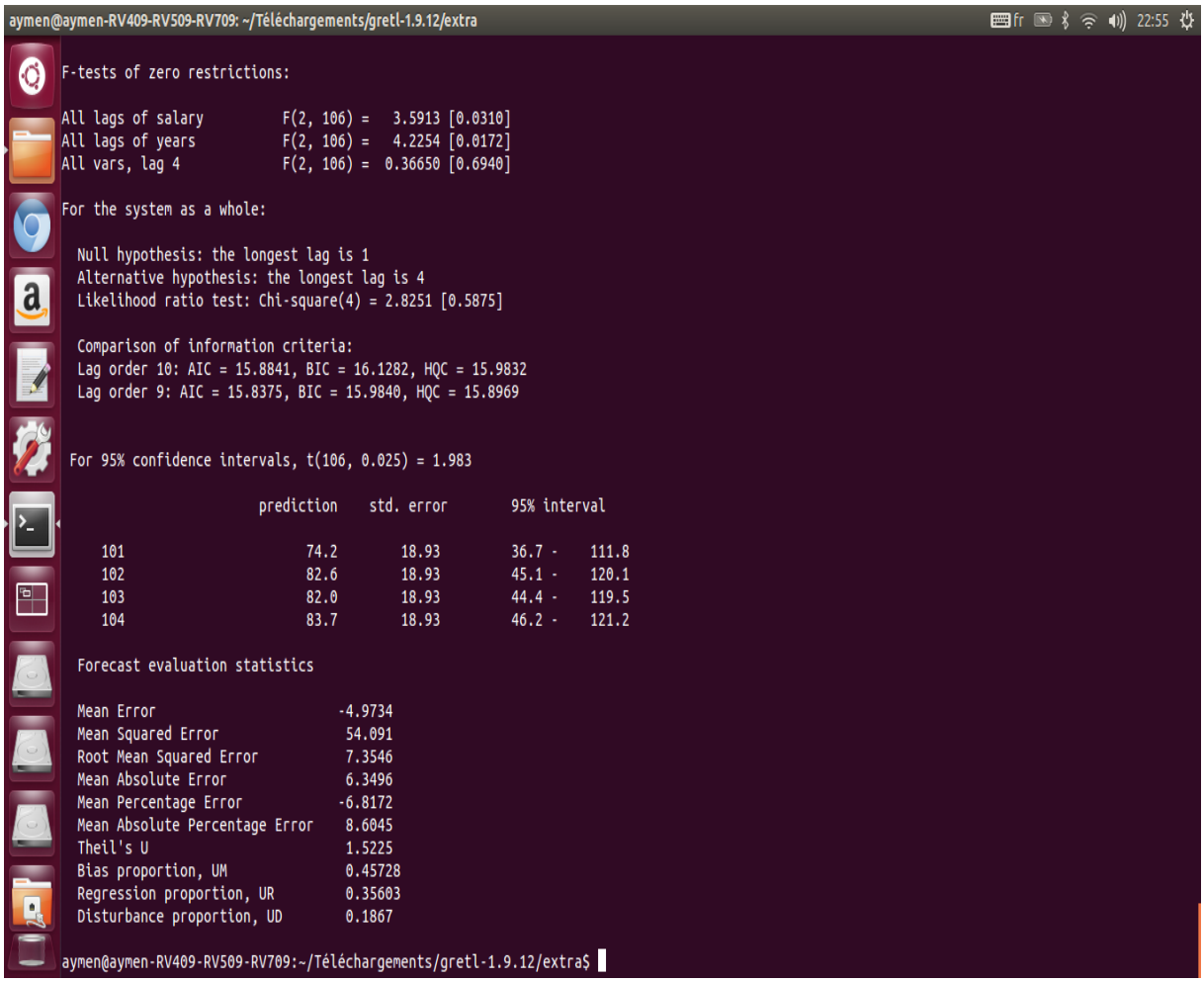


FIGURE 5.10 – La prédiction des valeurs futures

5.2.3 Plus court temps de parcours

Cette partie constitue le cœur du notre projet. Le principe est d'entrer deux points géographiques distants avec une date pour exécuter de l'algorithme de plus court temps de parcours. La figure 5.11 illustre une page web qui communique avec notre service pour le tester. Cette page web est hébergé par la la société CLASS&CO SA en prenant comme nom de marque MARCEL. La page web est composée de quatre champs de texte, le premier est celui de l'origine, le second est celui de destination, le troisième est celui de la date et le dernier est celui de l'heure. Ces champs sont munis de la fonction d'auto complétion.

La figure 5.12 illustre la page de réponse à une réservation d'un VTC. L'origine de cette réservation est Hôtel Saint joseph et la destination est Théâtre Mogador et la date le 27/06/2014 à 4 :10. Le client s'informe à la durée du trajet et le chemin du trajet et la distance.

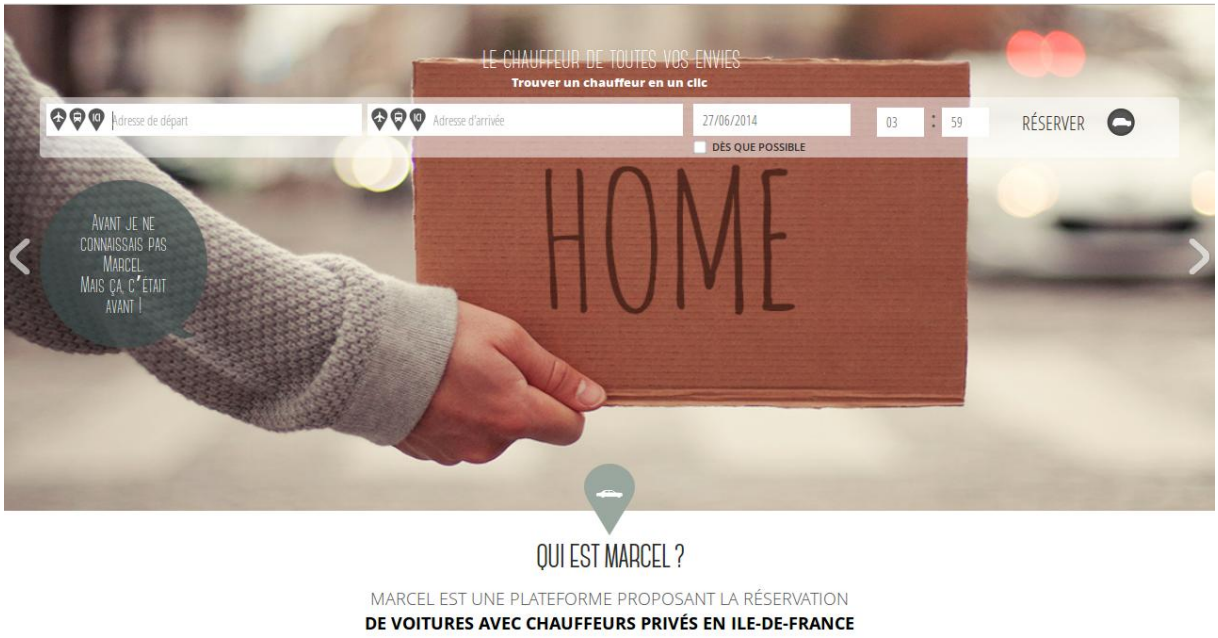


FIGURE 5.11 – Page web de réservation d'un VTC

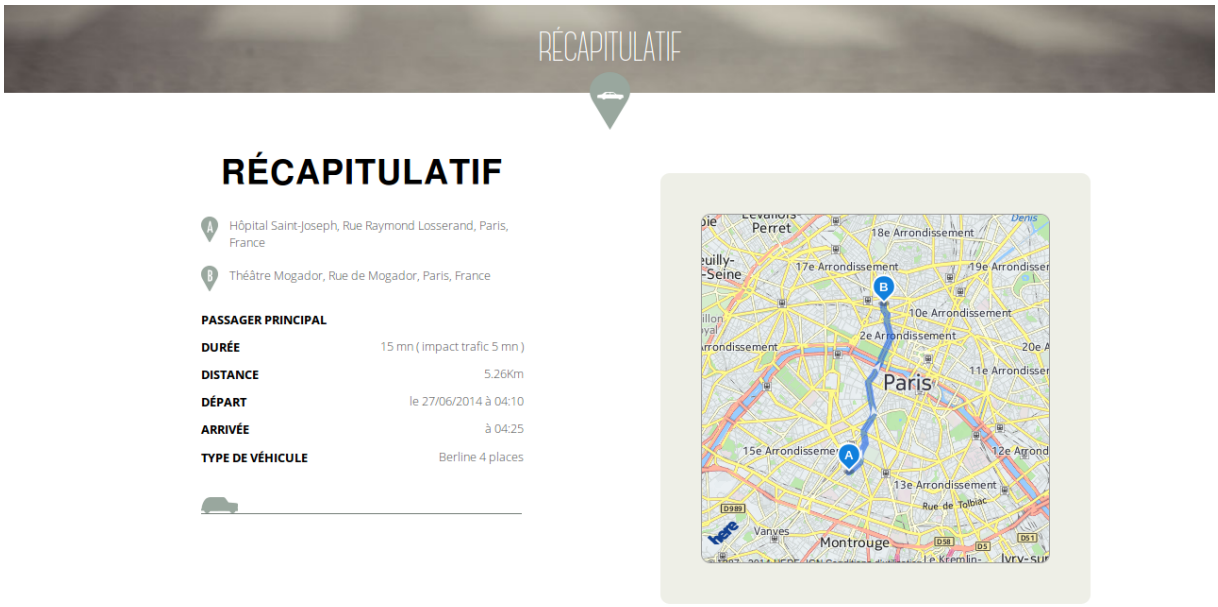
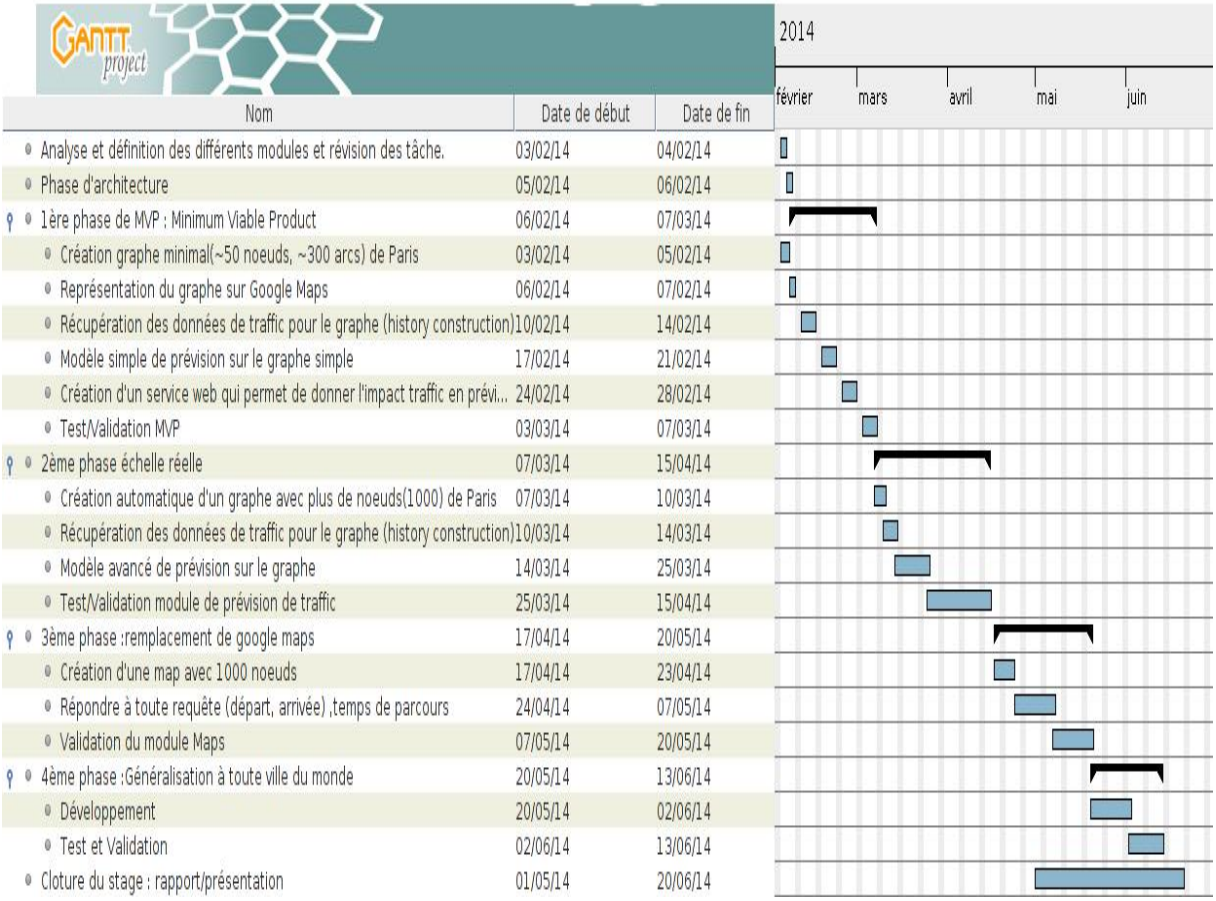


FIGURE 5.12 – Détail de la résercation

5.2.4 Gestion du projet

La gestion de projet a été une partie relativement importante de notre projet. Nous y avons consacré du temps au début de stage afin de planifier nos tâches et nous avons chaque fin de semaine tenu compte de notre avancement et ré-estimé nos charges. La Figure 5.13 exhibe la répartition de temps entre les différentes tâches du projet.



Nom	Date de début	Date de fin
Analyse et définition des différents modules et révision des tâche.	03/02/14	04/02/14
Phase d'architecture	05/02/14	06/02/14
1ère phase de MVP : Minimum Viable Product	06/02/14	07/03/14
Création graphe minimal(~50 noeuds, ~300 arcs) de Paris	03/02/14	05/02/14
Représentation du graphe sur Google Maps	06/02/14	07/02/14
Récupération des données de traffic pour le graphe (history construction)	10/02/14	14/02/14
Modèle simple de prévision sur le graphe simple	17/02/14	21/02/14
Création d'un service web qui permet de donner l'impact traffic en prévi...	24/02/14	28/02/14
Test/Validation MVP	03/03/14	07/03/14
2ème phase échelle réelle	07/03/14	15/04/14
Création automatique d'un graphe avec plus de noeuds(1000) de Paris	07/03/14	10/03/14
Récupération des données de traffic pour le graphe (history construction)	10/03/14	14/03/14
Modèle avancé de prévision sur le graphe	14/03/14	25/03/14
Test/Validation module de prévision de traffic	25/03/14	15/04/14
3ème phase :remplacement de google maps	17/04/14	20/05/14
Création d'une map avec 1000 noeuds	17/04/14	23/04/14
Répondre à toute requête (départ, arrivée) ,temps de parcours	24/04/14	07/05/14
Validation du module Maps	07/05/14	20/05/14
4ème phase :Généralisation à toute ville du monde	20/05/14	13/06/14
Développement	20/05/14	02/06/14
Test et Validation	02/06/14	13/06/14
Cloture du stage : rapport/présentation	01/05/14	20/06/14

FIGURE 5.13 – Chronogramme des Tâches.

Conclusion

Dans ce dernier chapitre, nous avons présenté l'environnement du travail ainsi que les choix techniques adoptés lors de l'implémentation. Nous avons également exposé le fonctionnement de notre système par grilles d'écrans comme résultat de notre étude tout au long de ce rapport. Enfin nous avons établi un chronogramme du projet détaillant son évolution.

Conclusion et Perspectives

L'objectif principal de ce projet était détudier la faisabilité, de concevoir et de réaliser un service web qui permet davoir pour un trajet donné et une date future donnée une estimation du temps de parcours avec trafic. En effet ce projet comporte trois parties : la première partie consiste à mettre en place un graphe d'une ville d'une façon automatique et la deuxième partie consiste à développer un outil de prédiction du trafic en se basant sur l'historique et la troisième partie développement d'un service web de calcul du plus court temps de parcours.

La première étape dans la réalisation de notre projet, était de faire une étude théorique sur les services web de web mapping et les problèmes de cheminement et finalement sur la prédiction. Ensuite nous avons entamé l'analyse et la spécification des besoins et nous avons pu dégager les besoins fonctionnels et non fonctionnels de notre projet. Partant de la spécification nous avons défini l'architecture de notre système, et nous avons commencé la conception de l'application à travers les différents diagrammes à savoir les diagrammes de classe et les diagrammes de séquence et les diagrammes d'architecture. Le reste du travail consistait à implémenter l'application en utilisant les bibliothèques Boost et Gretl et plusieurs autres.

Ce projet de fin d'étude était une expérience enrichissante et fructueuse qui nous a permis d'acquérir un ensemble de compétences sur divers plans. D'une part, sur le plan théorique, il nous a permis de nous familiariser avec le monde des statistiques et de prédiction et les services de VTC. D'autre part, sur le plan pratique, il nous a permis de mettre à l'épreuve nos connaissances pratiques en matière de conception et de développement et d'approfondir nos connaissances sur le développement sur une plateforme logicielle libre et événementielle en JavaScript.

Il reste encore des choses à faire, et comme perspectives de notre travail, nous proposons d'ajouter d'autres fonctionnalités telles que la fonction de trafic en temps réel. De plus, nous proposons de permettre de généraliser l'utilisation de cette application dans plusieurs pays non seulement la ville de Paris.

Bibliographie

[B1] V. Vapnik, “statistical learning theory”, John Wiley, New York, 1998.

[B2] K. Fukunaga, “Introduction to statistical pattern recognition”, Academic Press, New York,1972.

[B3] M. Kearns, U. Vazirani, “An introduction to computational learning theory”, MIT Press, Cambridge, Massachusetts, 1994.

[B4] Olivier Bousquet, Stéphanophore, Gabor Lugosi, “Introduction to statistical learning theory”

[B5] Isabelle Guyon, Andre Elisseeff, “An introduction to variable and feature selection”. Journal of Machine Learning Research, 3 :1157–1182, 2003. ISSN 1533-7928.

[B6] SelimGurun, Chandra Krintz, and Rich Wolski, “NWSLite : A General-Purpose, Nonparametric Prediction Utility for Embedded Systems”, ACM Transactions on Embedded Computing Systems, Vol. 7, No. 3, Article 32, Avril 2008.

[B7] Jian Zhang, Renato J. Figeiredo, “Learning-aided predictor integration for system performance prediction”, Cluster Comput (2007) 10 : 425—442, Springer Science+Business Media, LLC 2007.

[B8] ahmed ammari, “Conception et réalisation d’une application de webmapping d’analyse territoriale sur des SIG et bases de données open source : cas du territoire camerounais

Webographie

[N1] http://fr.wikipedia.org/wiki/Cartographie_en_ligne , *Dernière visite le 9/03/2014.*

[N2] http://maps.google.fr/enterprise/mapsearth/products/mapsapi.html *Dernière visite le 9/04/2014.*

[N3] http://www.coderscloud.com/team-top, *Dernière visite le 9/05/2014.*

[N4] http://www.challenges.fr/high-tech/20131210.CHA8145/
uber-la-start-up-qui-veut-revolutionner-le-transport-en-ville-a-paris-comme-a-sa
html , *Dernière visite le 3/05/2014.*

[N5] http://lexpansion.lexpress.fr/entreprises/la-societe-de-vtc-lecab-prend-les-taxis
_1511550.html , *dernière visite le 22/03/2014*

[N6] http://developer.nokia.com/community/wiki/HERE_Maps_API, *dernière visite le 10/04/2014*

[N7] www.polytech.univ-savoie.fr/uploads/media/CoursGraphe.pdf, *dernière visite le 27/05/2014*

[N8] http://www.nimbustier.net/publications/djikstra/djikstra.html, *dernière visite le 23/05/2014*

[N9] www.dil.univ-mrs.fr/~gcolas/algo-licence/slides/BF-slides.pdf, *dernière visite le 23/05/2014*

[N10] `http://webdocs.cs.ualberta.ca/~mmueller/ps/hpastar.pdf`, *dernière visite le 23/05/2014*

Annexe A

Problèmes de cheminement

A.1 Algorithme de Dijkstra

Cette annexe est tirée du site `http://www.apprendre-en-ligne.net/graphes/dijkstra/algorithme.html`. Il existe de nombreux liens sur Internet relatifs à cet algorithme.

Edgser Wybe Dijkstra (1930-2002) a proposé en 1959 un algorithme qui permet de calculer le plus court chemin entre un sommet particulier et tous les autres. Le résultat est une arborescence.

A.1.1 Principe de l'algorithme

Numérotons les sommets du graphe G = (V,E) de 1 à n. Supposons que l'on s'intéresse aux chemins partant du sommet 1. On construit un vecteur l = (l(1) ; l(2) ; ... ; l(n)) ayant n composantes tel que l(j) soit égal à la longueur du plus court chemin allant de 1 au sommet j. On initialise ce vecteur à c1,j, c'est-à-dire à la première ligne de la matrice des coûts du graphe, définie comme indiqué ci-dessous : où d(i,j) est le poids (la longueur) de l'arc (i,j). Les cij doivent être strictement positifs. On construit un autre vecteur p pour mémoriser le chemin pour aller du sommet 1 au sommet voulu. La valeur p(i) donne le sommet qui précède i dans le chemin. On considère ensuite deux ensembles de sommets, S initialisé à 1 et T initialisé à 2, 3, ..., n. A chaque pas de l'algorithme , on ajoute à S un sommet jusqu'à ce que S = V de telle sorte que le vecteur l donne à chaque étape le coût minimal des chemins de 1 aux sommets de S.

A.1.2 Description de l'algorithme

On suppose ici que le sommet de départ (qui sera la racine de l'arborescence) est le sommet 1. Notons qu'on peut toujours renuméroter les sommets pour que ce soit le cas.

Initialisations : l(j) = c1,j et p(j) = NIL, pour 1 j n
Pour 2 j n faire
Si c1,j < alors p(j) = 1.
S = 1 ; T = 2, 3, ..., n.
Iterations : Tant que T n'est pas vide faire
Choisir i dans T tel que l(i) est minimum
Retirer i de T et líajouter a S
Pour chaque successeur j de i, avec j dans T, faire
Si l(j) > l(i) + d(i,j) alors
l(j) = l(i) + d(i,j)
p(j) = i

A.2 Algorithme de Bellman-Ford

Cette annexe est tirée du site `http://www.apprendre-en-ligne.net/graphes/bell-man/algorithme.html`. Il existe de nombreux liens sur Internet relatifs à cet algorithme.

L'algorithme de Bellman-Ford (Bell-End-Ford) (Richard Bellman, Samuel End et Lester Ford) est un algorithme de programmation dynamique qui permet de trouver des plus courts chemins, depuis un sommet source donné, dans un graphe orienté pondéré. Contrairement à l'algorithme de Dijkstra, qui ne peut être utilisé que lorsque tous les arcs ont des poids positifs ou nuls.

A.2.1 Principe de l'algorithme

Ce premier algorithme est applicable sur tous les types de graphes, il détectera même sil y a un cycle absorbant ou non. Lidée est de parcourir tous les sommets et deectuer la technique de relâchement jusquà ce que les distances calculées soient stabilisées, i.e. quon ne puisse plus les améliorer. Pour cela, nous verrons quau pire il faudra parcourir n 1 fois tous les arcs.

A.2.2 Description de l'algorithme

Bellman-Ford(G(S,A,v),x0)
1 : Initialisation(G,x0) 2 : pour k = 1 à n 1 faire 3 : pour chaque arc (i, j) A faire 4 : Relâcher(G,i,j) 5 : n pour 6 : n pour 7 : pour chaque arc (i, j) A faire 8 : si (j) > (i) + v(i, j) alors 9 : retourner FAUX et sortir de Bellman-Ford 10 : nsi 11 : n pour

A.3 algorithme A étoile

Au premier abord, on pourrait se dire que pour trouver un chemin d'un point à un autre il faut commencer par se diriger vers la destination. Et bien ... c'est justement cette idée qu'utilise l'algorithme A*. L'idée est très simple : à chaque itération (oui, c'est un algorithme itératif), on va tenter de se rapprocher de la destination, on va donc privilégier les possibilités directement plus proches de la destination, en mettant de côté toutes les autres.

Toutes les possibilités ne permettant pas de se rapprocher de la destination sont mises de côté, mais pas supprimées. Elles sont simplement mises dans une liste de possibilités à explorer si jamais la solution explorée actuellement s'avère mauvaise. En effet, on ne peut pas savoir à l'avance si un chemin va aboutir ou sera le plus court. Il suffit que ce chemin amène à une impasse pour que cette solution devienne inexploitable.

L'algorithme va donc d'abord se diriger vers les chemins les plus directs. Et si ces chemins n'aboutissent pas ou bien s'avèrent mauvais par la suite, il examinera les solutions mises de côté. C'est ce retour en arrière pour examiner les solutions mises de côté qui nous garantit que l'algorithme nous trouvera toujours une solution (si tenté qu'elle existe, bien sûr).

On peut donc lui donner un terrain avec autant d'obstacles qu'on veut, aussi tordus soient-ils, s'il y a une solution, A* la trouvera.

A.3.1 Les listes A étoile

A* utilise deux listes, ces listes contiennent des points. Pour être plus général, on peut même dire que ces listes contiennent des noeuds d'un graphe, lequel graphe représentant notre terrain.

La première liste, appelée liste ouverte, va contenir tous les noeuds étudiés. Dès que l'algorithme va se pencher sur un noeud du graphe, il passera dans la liste ouverte (sauf s'il y est déjà).

La seconde liste, appelée liste fermée, contiendra tous les noeuds qui, à un moment où à un autre, ont été considérés comme faisant partie du chemin solution. Avant de passer dans la liste fermée, un noeud doit d'abord passer dans la liste ouverte, en effet, il doit d'abord être étudié avant d'être jugé comme bon.

A.3.2 Déroulement de l'algorithme

Pour déterminer si un noeud est suceptible de faire partie du chemin solution, il faut pouvoir quantifier sa qualité. Vous êtes entièrement libre de ce côté là, vous pouvez mesurer la pertinence d'un noeud de la manière qui vous plait. Néanmoins, une méthode

souvent utilisée et qui donne de bons résultats est de mesurer l'écartement entre ce noeud et le chemin à vol d'oiseau. On calcule donc la distance entre le point étudié et le dernier point qu'on a jugé comme bon. Et on calcule aussi la distance entre le point étudié et le point de destination. La somme de ces deux distances nous donne la qualité du noeud étudié. Plus un noeud à une qualité faible, meilleur il est.

Vous pouvez calculer ces distances de la manière que vous voulez, distance euclidienne, distance de Manhattan ou autre, elles peuvent convenir.

On ne sait pas grand chose du chemin solution si ce n'est que le point qui pourra nous rapprocher de la solution est un point voisin du point que nous étudions. On va donc étudier chacun des noeuds voisins du noeud courant pour déterminer celui qui a le plus de chances de faire partie du chemin solution.

La recherche du chemin commence par le premier point, en étudiant tous ses voisins, en calculant leur qualité, et en choisissant le meilleur pour continuer. Chaque point étudié est mis dans la liste ouverte et le meilleur de cette liste passe dans la liste fermée, il va servir de base pour la recherche suivante.

Ainsi, à chaque itération on va regarder parmi tous les noeuds qui ont été étudiés (et qui n'ont pas encore été choisis) celui qui a la meilleure qualité. Et il est tout à fait possible que le meilleur ne soit pas un voisin direct du point courant. Cela signifira que le point courant nous éloigne de la solution et qu'il faut corriger le tir.

L'algorithme s'arrête quand la destination a été atteinte ou bien lorsque toutes les solutions mises de côté ont été étudiées et qu'aucune ne s'est révélée bonne, c'est le cas où il n'y a pas de solution.